

On-the-Fly Temporal-Logic Task Allocation for Heterogeneous Multi-Robot Systems during Büchi Automaton Generation

IEEE Publication Technology, *Staff, IEEE,*

Abstract—This paper studies multi-robot task allocation under global linear temporal logic (LTL) specifications for heterogeneous robot teams. Existing methods typically construct the complete automaton before performing task allocation, which can delay the first feasible solution in large-scale problems and hinder rapid plan repair when robot availability changes during execution. To address these limitations, we propose an on-the-fly framework that performs temporal-logic task allocation within Büchi automaton generation. The method prunes infeasible and redundant transitions during generalized Büchi automaton construction, and then synchronously builds a nondeterministic Büchi automaton and a plan graph. Once an accepting prefix-suffix structure is reached, the framework can immediately return an initial executable plan. After the plan graph is fully constructed, a feasible solution extracted from the completed graph is used to warm-start a branch-and-bound optimizer for further improvement. The same workflow is also used to repair the remaining mission when robots become unavailable during execution. Theoretical analysis establishes completeness of the proposed framework. The proposed framework is validated through extensive numerical simulations against strong baselines and hardware experiments. The results demonstrate that it generates feasible plans much faster in large-scale systems with complex LTL specifications, while also maintaining rapid plan repair when robots become unavailable during online execution.

Index Terms—Linear temporal logic, multi-robot task allocation, heterogeneous robots, anytime search.

I. INTRODUCTION

Multi-robot systems (MRS) have been widely used in applications such as persistent surveillance [1], logistics [2], and autonomous maintenance [3], where multiple robots can operate concurrently and cooperatively to improve efficiency and accomplish tasks that are beyond the capability of a single robot. In practice, many multi-robot tasks are subject to temporal requirements. For instance, in a smart factory or large warehouse, carriers may first transport tools or shelves to a target region, inspection robots may verify safety conditions before manipulation begins, and monitoring robots may be required to continuously patrol surrounding areas. Such task dependencies naturally introduce temporal constraints such as ordering, persistence, and synchronization. To specify these constraints in a rigorous way, many recent work have used formal languages such as linear temporal logic (LTL) [4]; see, for example, [5], [6].

In this work, we consider the multi-robot task allocation (MRTA) problem under temporal logic specifications, referred to as LTL-MRTA, which was formalized in [7]. In LTL-MRTA, the task is specified by an LTL formula, but the specific robots assigned to each subtask are not predetermined by the formula. For example, a specification may require two type-1 robots to complete a pickup-and-delivery task before waiting at the delivery site until a type-2 robot arrives, without prescribing which individual robots should execute these subtasks. This problem becomes increasingly difficult as the number of robots and the complexity of the task specification grow, since the associated search and optimization space may expand explosively. As a result, existing approaches, such as mixed-integer linear programming (MILP)-based methods [7], [8] and search-based methods [9], [10] can be computationally demanding in large-scale problems, even when only an initial feasible solution is required.

In practical robotic applications, however, it is often important to obtain an executable solution early, especially in large-scale settings where task execution needs to start promptly and decisions may need to be updated online. Planning-tree-based [6], [11] and partial-order-based methods [3] highlighted the practical importance of fast first-solution generation. However, planning-tree-based methods may be prone to local optima, while partial-order-based methods may be less effective for complex tasks due to the need to extract the partial-order relations among subtasks. Consequently, for LTL-MRTA, developing methods that can quickly return a feasible solution and further improve it toward optimality remains an important research direction. In real-robot execution, the available team may also change unexpectedly due to hardware faults, robot damage, localization failures, or battery depletion. Such events do not necessarily change the temporal-logic mission itself, but they invalidate assignments involving the unavailable robots and require the remaining mission to be repaired from the current execution state. Therefore, an LTL-MRTA planner should not only generate an executable plan before execution, but also support fast online replanning when the available robot set changes.

To address this problem, we propose a novel framework that integrates robot planning into the automaton generation process. Unlike existing methods that first construct the complete NBA and then perform planning on top of it, our approach introduces information about the robot team, the workspace, and intermediate planning results during the translation from LTL to automata. Specifically, we follow the translation pipeline of LTL2BA [12], a classical tool widely used in the literature [13], [14], which translates an LTL formula into a very weak Büchi automaton (VWAA), then into a generalized Büchi

This paper was produced by the IEEE Publication Technology Group. They are in Piscataway, NJ.

Manuscript received April 19, 2021; revised August 16, 2021.

automaton (GBA), and finally into a nondeterministic Büchi automaton (NBA). During GBA construction, each candidate transition is screened before insertion against the robot types and cardinalities available in the team, and candidates whose simultaneous action requirements cannot be satisfied are discarded. After the GBA is constructed, transitions that impose unnecessary synchronization are removed. During NBA generation, each newly generated transition induces a subtask. A greedy task-allocation procedure is then applied to the induced subtask. The feasible assignment is recorded in a plan graph. Consequently, an executable plan becomes available as soon as an accepting prefix-suffix structure is generated, before the complete NBA has been constructed. After the plan graph has been completed, the executable plan initializes the incumbent of a branch-and-bound optimizer. When a robot becomes unavailable during execution, the method re-roots the retained plan graph at the last confirmed execution state and re-optimizes the unfinished assignments over the surviving team. In this way, the proposed method supports rapid first-solution generation as well as continued improvement toward better solutions.

A. Related Work

Existing planning methods under LTL constraints can generally be classified into bottom-up and top-down approaches. Bottom-up approaches usually formulate the problem as a task-coordination problem under local LTL specifications, where each robot is assigned a local LTL task and team behavior is achieved through communication, synchronization, and online replanning. For example, Guo et al. [15] studied a decentralized planning framework under local LTL specifications with online plan reconfiguration, while Yu et al. [16] developed a distributed motion-coordination strategy under local LTL specifications, where conflicts among neighboring robots are detected and resolved online to ensure safety and task satisfaction. In contrast, top-down approaches start from a global LTL formula that specifies the overall team mission. This paper focuses on the latter setting.

For top-down methods, a common strategy is to construct the product automaton of NBA translated from the global LTL formula and the discrete transition systems of the robots and then solve the resulting graph-search problem [9], [17]. However, this approach suffers from state explosion, because the number of states in the product automaton increases rapidly with the workspace/state-space size, the number of robots, and the complexity of the task, as reflected by the size of the automaton derived from the specification. To alleviate this issue, sampling-based methods [10], [18] incrementally construct trees that approximate the state space and transitions of the synchronous product automaton, instead of explicitly building the full product automaton. This significantly improves scalability while retaining probabilistic completeness and asymptotic optimality. However, such methods typically require the global LTL formula to explicitly assign subtasks to robots. When the robot assignment is not given a priori, a possible workaround is to enumerate all admissible robot combinations and connect them using OR operators, but this can result in exponentially long LTL formulas.

To avoid the combinatorial burden and reduced flexibility caused by explicitly allocate tasks to robots, some works decompose the global LTL specifications. For example, [14], [19] proposed the simultaneous task allocation and planning (STAP) framework, which constructs a team model that integrates task decomposition and task allocation. By decomposing the global task into parallelizable subtasks, the framework improves planning efficiency. However, these works do not explicitly address tightly coupled collaborative tasks requiring the simultaneous participation of multiple robots. Other works address global temporal-logic task planning by using mixed-integer linear programming (MILP) [7], [8], [20]. In particular, to mitigate the impact of workspace size on the computational burden, [7] proposed a hierarchical architecture that separates task planning from motion planning. Unfortunately, MILP-based methods can become computationally demanding in large-scale settings, even when only a first feasible solution is required.

Planning-decision-tree-based methods [11], [21] have been proposed to improve the speed of temporal-logic mission planning. These methods represent task progression and task allocation incrementally, enabling real-time planning for large-scale multi-robot systems. Although such methods are highly efficient in generating feasible solutions, they typically rely on greedy task-allocation decisions, and therefore can be prone to getting trapped in local optima. Liu et al. [3] analyze the task automaton to extract partial-order relations among subtasks, and then use a branch-and-bound search procedure to compute task assignments. Their method explicitly emphasizes the importance of obtaining a valid plan early: it is formulated as an anytime algorithm, so that a feasible solution can be returned quickly and then progressively improved as computation continues. However, when the task specification becomes more complex, the efficiency of obtaining the first feasible solution may decrease, since extracting the partial-order relations among subtasks can itself take additional time. As illustrated in the paper, for an automaton with hundreds of states, the algorithm may require tens or even hundreds of seconds to obtain the first partial order. Some work reduces the burden of complex specifications by changing or exploiting the structure of the task formula. Hierarchical temporal-logic specifications [22] organize complex specifications into a hierarchy. Liu et al. [23] exploit the case where an sc-LTL mission is given as a conjunction of subformulas; each subformula is converted into a poset and the products of these posets are computed online to accommodate newly released tasks. These methods reduce the burden of complex specifications mainly by changing or exploiting the modeled structure of the LTL specification. While effective, this perspective may require the mission to be expressed in a suitable hierarchical, conjunctive, or otherwise structured form, and shift part of the complexity to specification design or task decomposition.

Overall, while existing top-down methods have advanced LTL-MRTA from different perspectives, they still struggle to simultaneously achieve rapid first-solution generation and effective subsequent optimization in large-scale problems. This limitation is particularly evident when substantial computation is required before a planning structure suitable for allocation

can even be obtained. Instead of constructing the complete NBA first and then solving the allocation problem on top of it, the proposed method incorporates robot-team feasibility, workspace travel information, and partial planning results into the LTL-to-automaton translation process. During GBA construction, infeasible or decomposition-inducing transitions are pruned before they are propagated to the NBA stage. During the NBA construction stage, the NBA and a shared directed planning graph are generated synchronously. Whenever a new automaton transition becomes available, the corresponding transition is evaluated by a greedy executable assignment, and newly discovered successor NBA states are ordered within the current pending batch by a lightweight residual-obligation score. Once an accepting prefix-suffix structure is reached, an executable initial plan can be extracted and used as a warm start for the subsequent branch-and-bound assignment optimizer. When robot failures occur during execution, the proposed method can rapidly repair the plan for the remaining mission. The proposed framework supports both fast initial planning and continued improvement toward higher-quality solutions.

B. Contributions

We propose a new on-the-fly approach to LTL-MRTA that performs task allocation as the NBA is generated. Unlike conventional methods that construct the complete NBA before allocation, our method performs task allocation incrementally using a greedy procedure during NBA construction. During this process, feasible partial assignments and their execution states are recorded in a plan graph. This graph grows synchronously with the NBA. An executable plan becomes available as soon as an accepting structure is reached, even if the NBA is not yet complete. One innovation of our approach is that the GBA is pruned before NBA construction begins. We discard transitions that are infeasible for the available robot team or impose avoidable synchronization. This reduces the number of transitions passed to the NBA stage. Another distinctive aspect is that a residual-obligation score guides both the NBA and the plan graph as they are constructed. The score reflects the current plan cost and the estimated travel needed for the remaining tasks. Rather than following a DFS order used in standard LTL2BA construction, our method gives priority to Büchi successors with lower scores. The NBA and the plan graph are constructed together. Thus, the same ordering also guides the exploration of task-allocation branches. More promising branches are explored earlier. This helps the method obtain a high-quality initial plan sooner. To the best of our knowledge, this is the first LTL-MRTA framework to perform task allocation during score-guided NBA construction.

The rest of the article is organized as follows. Sections II and III present the preliminaries and formulate the LTL-MRTA problem, respectively. Section IV provides an overview of the proposed framework. Section V introduces the GBA-level pruning, and Section VI details the construction of the NBA and the plan graph. Section VII presents the branch-and-bound optimization, while Section VIII describes plan adaptation

under robot unavailability. Sections IX and X provide the complexity and theoretical analyses, respectively. Section XI and XII report the numerical and physical experiments, and Section XIII concludes the article.

II. PRELIMINARIES

A. Linear Temporal Logic

Linear temporal logic (LTL) is widely used to specify temporal tasks over a set of atomic propositions AP . Following the standard syntax, an LTL formula over AP is recursively defined by

$$\varphi := \top \mid \pi \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid \bigcirc\varphi \mid \varphi_1 \mathcal{U} \varphi_2,$$

where $\pi \in AP$, $\top$ denotes the constant true, $\bigcirc$ is the next operator, and $\mathcal{U}$ is the until operator. Other temporal operators can be derived in the standard way, including eventually $\Diamond\varphi := \top \mathcal{U} \varphi$, always $\Box\varphi := \neg\Diamond\neg\varphi$.

Let $\Sigma = 2^{AP}$ be the alphabet. An infinite word over Σ is a sequence $w = \sigma_0\sigma_1 \dots \in \Sigma^\omega$, where each $\sigma_k \in \Sigma$ denotes the set of atomic propositions satisfied at step k . The notation $w \models \varphi$ means that the word w satisfies the LTL formula φ . The language of φ is defined as $\text{Words}(\varphi) := \{w \in \Sigma^\omega \mid w \models \varphi\}$.

It is well known that any satisfiable LTL formula can be translated into a nondeterministic Büchi automaton (NBA).

Definition 1 (Nondeterministic Büchi Automaton). A nondeterministic Büchi automaton (NBA) is a tuple

$$\mathcal{B}_\varphi = (Q_B, Q_{B,0}, \Sigma_B, \rightarrow_B, Q_{B,F}), \quad (1)$$

where Q_B is a finite set of states, $Q_{B,0} \subseteq Q_B$ is the set of initial states, Σ_B is the alphabet, $\rightarrow_B \subseteq Q_B \times \Sigma_B \times Q_B$ is the transition relation, and $Q_{B,F} \subseteq Q_B$ is the set of accepting states.

An infinite run of $\mathcal{B}$ over a word $w = \sigma_0\sigma_1 \dots$ is a sequence $\rho = q_0q_1 \dots$ such that $q_0 \in Q_{B,0}$ and $(q_k, \sigma_k, q_{k+1}) \in \rightarrow_B$ for all $k \geq 0$, where each $\sigma_k \in \Sigma_B$. The run is accepting if it visits accepting states infinitely often, i.e., $\text{Inf}(\rho) \cap Q_{B,F} \neq \emptyset$. For a satisfiable LTL formula, an accepting run can be written in the prefix-suffix form, where the prefix reaches an accepting state once and the suffix repeats a cycle around that accepting state indefinitely.

B. LTL2BA Translation

In this work, we follow the translation pipeline of LTL2BA [12] to construct automata from LTL specifications. Given an LTL formula φ , LTL2BA first rewrites φ into negative normal form (NNF), so that negation only appears in front of atomic propositions. It then applies rewriting rules to simplify the formula and reduce the number of temporal operators before automaton generation. An abstract syntax tree is then generated from the simplified formula.

Definition 2 (Abstract Syntax Tree). Given an LTL formula φ in negative normal form, its abstract syntax tree (AST), denoted by $\mathcal{T}_\varphi$, is a rooted ordered tree in which each internal node is labeled by a logical or temporal operator, and each leaf

is labeled by an atomic proposition or its negation. The recursive structure of $\mathcal{T}_\varphi$ represents the syntactic decomposition of φ and provides the basis for extracting temporal subformulas during automaton construction.

Starting from the AST, LTL2BA first constructs a very weak alternating automaton (VWAA). An alternating automaton may require multiple successor states to hold simultaneously under the same input symbol. Moreover, the “very weak” property imposes a partial order on states, so that transitions can only remain at the same level or move downward in this order, which yields a simpler automaton structure.

Definition 3 (Very Weak Alternating Automaton). A very weak alternating automaton (VWAA) is a tuple

$$\mathcal{V}_\varphi = (Q_v, \Sigma, \delta_v, I_v, F_v, \preceq_v),$$

where Q_v is a finite set of states, Σ is the input alphabet, δ_v is the transition function, I_v is the initial condition, $F_v \subseteq Q_v$ is the set of accepting states, and $\preceq_v$ is a partial order on Q_v such that every state appearing in the transition of a state $q_v \in Q_v$ is lower than or equal to q_v under $\preceq_v$.

Based on the VWAA, LTL2BA then constructs a generalized Büchi automaton (GBA), which preserves multiple acceptance requirements in a compact intermediate form before the final degeneralization step.

Definition 4 (Generalized Büchi Automaton). A generalized Büchi automaton (GBA) is a tuple

$$\mathcal{G}_\varphi = (Q_g, Q_{g,0}, \Sigma, \rightarrow_g, \mathcal{F}_g),$$

where Q_g is a finite set of states, $Q_{g,0} \subseteq Q_g$ is the set of initial states, Σ is the input alphabet, $\rightarrow_g \subseteq Q_g \times \Sigma \times Q_g$ is the transition relation, and $\mathcal{F}_g = \{F_{g,1}, \dots, F_{g,m}\}$ is a finite collection of accepting transition sets with $F_{g,i} \subseteq \rightarrow_g$ for all $i \in \{1, \dots, m\}$. A run is accepting if, for every $F_{g,i} \in \mathcal{F}_g$, it traverses transitions in $F_{g,i}$ infinitely often.

The VWAA-to-GBA construction is briefly summarized here. Following [12], each GBA state $q_g \in Q_g \subseteq 2^{Q_v}$ is a set of VWAA states and is identified with their conjunction. For example, $q_g = \{q_{v,1}, \dots, q_{v,n}\} \equiv q_{v,1} \wedge \dots \wedge q_{v,n}$, which means that all states in q_g must hold simultaneously. To generate an outgoing transition from q_g , one transition is selected from each $\delta_v(q_{v,i})$. The selected transition conditions are conjoined, or equivalently, their sets of satisfying input symbols are intersected. Their successor-state sets are also combined to form a successor GBA state q'_g . Each consistent combination produces a candidate transition $e_g = (q_g, \sigma, q'_g)$, where σ denotes the combined transition condition. After the standard implication-based simplification inherited from LTL2BA, the retained candidate transitions are inserted into $\rightarrow_g$.

The co-Büchi acceptance condition of the VWAA is transferred to the transition-based acceptance condition of the GBA. In particular, each co-Büchi state $f \in F_v$ induces one accepting transition set in $\mathcal{F}_g$. This acceptance information is subsequently used during the GBA-to-NBA transformation.

After constructing the GBA, LTL2BA applies a standard degeneralization step to obtain the final nondeterministic Büchi automaton (NBA), denoted by $\mathcal{B}_\varphi$. Therefore, the overall translation pipeline can be summarized as $\varphi \rightarrow \mathcal{T}_\varphi \rightarrow \mathcal{V}_\varphi \rightarrow \mathcal{G}_\varphi \rightarrow \mathcal{B}_\varphi$.

A key advantage of LTL2BA is that simplifications are performed throughout the translation process. Besides simplifying the formula before automaton generation, the algorithm also simplifies intermediate automata on-the-fly by removing inaccessible states, eliminating implied transitions, and merging equivalent states whenever possible. These operations reduce the number of intermediate states and transitions and improve both time and memory efficiency. In this paper, we build on this translation pipeline and further incorporate planning-related information during automaton construction so that the generated automaton structure is better aligned with the downstream task-planning problem.

III. PROBLEM DESCRIPTION

A. Workspace and Heterogeneous Robot Team

Consider a bounded workspace $\mathcal{W}$ containing a finite set of mutually disjoint regions of interest $\mathcal{L} = \{l_1, l_2, \dots, l_{n_l}\}$. Let $\mathcal{O} = \{o_1, o_2, \dots, o_{n_o}\}$ denote the non-interested regions such as obstacles and forbidden regions. We assume that regions of interest do not overlap with each other and regions of non-interest, i.e., $l_i \cap o_j = \emptyset$, $\forall i \in \{1, \dots, n_l\}$, $\forall j \in \{1, \dots, n_o\}$, $l_i \cap l_j = \emptyset$, $\forall i \neq j$.

A team of heterogeneous robots is denoted by $\mathcal{R} = \{r_1, r_2, \dots, r_{n_r}\}$. The robots belong to a finite set of types $\mathcal{T} = \{1, 2, \dots, n_t\}$, where each type represents a specific capability, such as sensing, manipulation, or mobility. Each robot belongs to exactly one type. Let $\tau(r_k) \in \mathcal{T}$ denote the type of robot r_k , and let $\mathcal{R}_j = \{r_k \in \mathcal{R} \mid \tau(r_k) = j\}$ be the set of robots of type j . The position of robot r_k is denoted by $x(r_k) \in \mathcal{W}$, and its constant travel speed is denoted by $v(r_k) > 0$. For each robot $r_k \in \mathcal{R}$, its motion between regions is modeled by its own transition graph $\mathcal{G}_{r,k} = (V_{r,k}, \rightarrow_{r,k})$, $V_{r,k} \subseteq \mathcal{L}$ is the set of regions accessible to robot r_k and $\rightarrow_{r,k} \subseteq V_{r,k} \times V_{r,k}$ denotes the set of region-to-region transitions permitted for robot r_k . For any position $x \in \mathcal{W}$ and target region $l_i \in \mathcal{L}$, let $d_k(x, l_i)$ denote the shortest feasible distance that robot r_k must travel from x to reach l_i . In particular, $d_k(x(r_k), l_i)$ is the distance from the current position of r_k to l_i . We set $d_k(x, l_i) = +\infty$ if l_i is unreachable by r_k .

B. Task Specification

The temporal-logic specification is defined over a set of atomic propositions AP . Before formulating the temporal logic task, we introduce two classes of atomic propositions. (i) $\mathcal{P} \triangleq \{p_i \mid l_i \in \mathcal{L}\}$ contains region propositions, where p_i is true when at least one available robot is located at region l_i . (ii) Let a_i be an action proposition. For example, a_i can represent a group of robots visits a specified target region. Let $\mathcal{A} \triangleq \{a_1, a_2, \dots, a_{n_a}\}$ denote the set of all action propositions. Each $a_i \in \mathcal{A}$ is associated with an execution requirement $c(a_i) = (\tau_i, n_i, l_i)$, where $\tau_i \in \mathcal{T}$ is the required

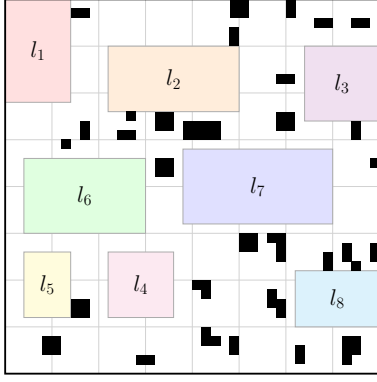


Fig. 1. Workspace of experiments.

robot type, $n_i \in \mathbb{N}$ is the required number of robots of type τ_i , and $l_i \in \mathcal{L}$ is the target region. The proposition a_i is true, i.e., $a_i = \top$, if and only if the corresponding action is performed by n_i robots of type τ_i at region l_i . In the factory scenario, an action proposition can represent a region-level operation such as inspection, material delivery, collaborative repair, or monitoring. The tuple $c(a)$ specifies the required robot capability, the number of robots that must participate, and the region where the operation must be completed. The complete set of atomic propositions used in the temporal logic specification is defined as $\mathcal{AP} \triangleq \mathcal{P} \cup \mathcal{A}$.

Example 1. For example, as shown in Fig.1 consider a large warehouse facility with eight regions of interest, denoted by $\mathcal{L} = \{l_1, l_2, \dots, l_8\}$. l_1 and l_2 are inbound receiving and temporary storage areas, l_3 and l_4 are order-picking and outbound loading areas, l_5 and l_6 are inspection and equipment-maintenance areas, and l_7 and l_8 are dispatch areas. The robot team consists of five robots, $\mathcal{R} = \{r_1, r_2, r_3, r_4, r_5\}$, where r_1, r_2, r_3 are type-1 robots and r_4, r_5 are type-2 robots. Consider a safety-inspection task $\Diamond(a_1 \wedge \Diamond(a_2 \wedge (a_3))) \wedge \Box \neg p_8$. Here, $c(a_1) = (2, 2, l_5)$, $c(a_2) = (1, 1, l_6)$ and $c(a_3) = (1, 1, l_1)$. This formula requires all robots to avoid the restricted dispatch area l_8 throughout execution. It also requires two type-2 robots to first perform a collaborative safety inspection at l_5 , after which one type-1 robot checks the equipment-maintenance area l_6 and another type-1 robot inspects the inbound receiving area l_1 .

C. Problem Definition

Consider an LTL formula φ over $\mathcal{AP}$. Let $B_\varphi = (Q_B, Q_{B,0}, \Sigma_B, \rightarrow_B, Q_{B,F})$ be the NBA associated with φ . Here, Q_B is the finite set of Buchi states, $Q_{B,0}$ is the set of initial states, $\Sigma_B = 2^{\mathcal{AP}}$ is the alphabet, $\rightarrow_B \subseteq Q_B \times \Sigma_B \times Q_B$ is the transition relation, and $Q_{B,F}$ is the set of accepting states. To connect the generated NBA structure with task-level planning, we associate each NBA transition with a subtask.

Definition 5 (Subtask). Consider an NBA transition $(q_i, \sigma_{ij}, q_j) \in \rightarrow_B$, where $q_i, q_j \in Q_B$ are NBA states. Let σ_i^s denote the self-loop condition at NBA state q_i . If no holding

condition is required, we set $\sigma_i^s = \top$. The subtask induced by this transition is defined as below.

$$\omega_{ij} = (\sigma_{ij}, \sigma_i^s). \quad (2)$$

Here, σ_{ij} specifies the propositions that enable the transition from q_i to q_j , while σ_i^s specifies the propositions that must be maintained before this transition is completed.

A plan of φ is defined as $\Pi = (\omega, \rho, \mathcal{C})$ where $\omega = \omega_0 \omega_1 \omega_2 \dots$ is the corresponding subtask sequence, $\rho = q_0 q_1 q_2 \dots$ is the corresponding sequence of NBA states, $\mathcal{C} = \mathcal{C}_0 \mathcal{C}_1 \mathcal{C}_2 \dots$ is the corresponding robot-assignment sequence. Specifically, for subtask ω_k , let $\mathcal{A}(\omega_k) \subseteq \mathcal{A}$ denote the set of action propositions that must be true at this transition. The assignment $\mathcal{C}_k : \mathcal{A}(\omega_k) \rightarrow 2^{\mathcal{R}}$ maps each required action proposition to the set of robots selected to execute it. Assignments for different action propositions in the same ω_k must be disjoint, i.e., $\mathcal{C}_k(a_i) \cap \mathcal{C}_k(a_j) = \emptyset$, $\forall a_i \neq a_j$, $a_i, a_j \in \mathcal{A}(\omega_k)$.

The accepting run of NBA can be written in a prefix-suffix structure. Thus, ρ can be written as $\rho = \rho_{\text{pre}}(\rho_{\text{suf}})^\omega$, where ρ_{pre} is the prefix run and ρ_{suf} is the suffix run. The corresponding plan Π admits the same prefix-suffix structure, i.e. $\Pi = \Pi_{\text{pre}}(\Pi_{\text{suf}})^\omega$. Here, Π_{pre} is the plan segment associated with ρ_{pre} , and Π_{suf} is the finite plan segment associated with one traversal of the suffix cycle ρ_{suf} . To represent one completed execution of the prefix-suffix plan, we define the finite plan $\Pi_{\text{fin}} = \Pi_{\text{pre}}\Pi_{\text{suf}}$. Let $T(\Pi_{\text{fin}})$ denote the completion time of Π_{fin} . The cost is defined as below.

$$J_\varphi = T(\Pi_{\text{fin}}). \quad (3)$$

Problem III.1. Consider a workspace $\mathcal{W}$ and a heterogeneous robot team $\mathcal{R}$ with type set $\mathcal{T}$, and a LTL formula φ defined over the atomic proposition set $\mathcal{AP} = \mathcal{P} \cup \mathcal{A}$. The goal is to develop a task plan for each robot such that the specification φ is satisfied and the cost in Eq. 3 is minimized.

IV. SYSTEM OVERVIEW

This section presents the proposed framework for LTL-MRTA, as summarized in Algorithm 1. The key idea is to integrate task-allocation reasoning into the LTL-to-automaton translation process, instead of constructing the complete NBA before planning. Specifically, the input LTL formula is first parsed into an abstract syntax tree and then translated into a VWAA following the LTL2BA pipeline. During the subsequent GBA construction, transitions that cannot be realized by the available robot team are pruned immediately. After the GBA is generated, decomposition-inducing transitions are further removed to avoid unnecessary synchronization among independent subtasks.

Given the pruned GBA, the framework constructs the NBA and a shared directed plan graph synchronously. Each newly generated NBA transition induces a subtask, which is evaluated on the fly using the robot assignments, predicted positions, and completion times stored in the current plan node. Infeasible successors are discarded, whereas feasible task progressions are inserted into the plan graph. Once an accepting prefix-suffix structure is reached, an initial feasible

plan is extracted. After the plan graph is completed, this feasible plan is used to warm-start a branch-and-bound optimizer, which further improves the completion time over the generated plan graph.

Algorithm 1 On-the-fly Task Allocation framework

Require: LTL formula φ , workspace $\mathcal{W}$, robot team $\mathcal{R}$, atomic propositions $\mathcal{AP}$.

Ensure: Feasible plan Π^* .

- 1: Parse φ and construct the VWAA $\mathcal{V}_\varphi$.
 - 2: Construct and prune the GBA $\mathcal{G}_\varphi^-$.
 - 3: Initialize the root plan node ν_0 .
 - 4: $(G_P, \Pi_{\text{init}}) \leftarrow \text{CONSTRUCTPLANGRAPH}(\mathcal{G}_\varphi^-, \mathcal{R}, \nu_0)$.
 - 5: $\Pi^* \leftarrow \text{BnB_Optimizer}(G_P, \Pi_{\text{init}})$.
 - 6: **return** Π^* .
-

V. PRUNING FOR GBA

In this section, we introduce the proposed pruning procedure during the construction of GBA. Unlike methods that perform pruning after the complete NBA has been constructed, the proposed method prunes the GBA both during and after the VWAA-to-GBA construction stage. This early pruning reduces the size of the GBA and prevents unnecessary transitions from being propagated to the subsequent NBA construction and planning stages. The proposed GBA pruning consists of the following two parts.

1) *Infeasible Transition Pruning During GBA Construction:* The simplifications performed during the construction of GBA rely on the logical formula and the evolving automaton structure, without incorporating information about the robot team or the workspace. We therefore introduce a robot-team feasibility check during GBA construction. Before a candidate transition $e_g = (q_g, \sigma_g, q'_g)$ is inserted into $\rightarrow_g$, the action requirements in σ_g are checked against the available robot team $\mathcal{R}$. If no subset of $\mathcal{R}$ can satisfy the requirements, e_g is infeasible for the considered system and is discarded immediately. Otherwise, it is retained for subsequent GBA construction. For example, suppose that a candidate transition requires three type-1 robots to perform an action at region l_1 and another three type-1 robots to perform an action at region l_2 simultaneously. If $\mathcal{R}$ contains only five type-1 robots, no feasible robot assignment can satisfy the transition condition. The candidate transition is therefore discarded before it is inserted into $\rightarrow_g$.

2) *Pruning of Progress-Consistent Decomposable Transitions:* The second pruning step is performed after the GBA has been constructed. Although pruning is applied at the GBA level, it targets unnecessary synchronization that would otherwise be inherited by the resulting NBA. Since each NBA transition induces a subtask, requiring several independent subtasks to be satisfied at the same transition can be unnecessarily restrictive [3]. The main challenge is to identify and remove such synchronization before the complete NBA is available. A GBA transition can therefore be removed only when decomposing its combined task requirements preserves the acceptance progress used in the subsequent GBA-to-NBA degeneralization. To formalize this decomposition without

restricting it to a particular automaton type, we first introduce a general definition of transition decomposability. The definition applies to both NBA and GBA transitions.

Definition 6 (Decomposable transition). Consider an automaton with state set Q , alphabet Σ , and transition relation $\rightarrow \subseteq Q \times \Sigma \times Q$. A transition $e_{ij} = (q_i, \sigma_{ij}, q_j) \in \rightarrow$ is called decomposable if there exist a state $q_k \in Q$ and two transitions $(q_i, \sigma_{ik}, q_k) \in \rightarrow$ and $(q_k, \sigma_{kj}, q_j) \in \rightarrow$ such that $\sigma_{ij} = \sigma_{ik} \cup \sigma_{kj}$.

Definition 6 applies to both NBA and GBA transitions. For GBA pruning, however, decomposability alone is not sufficient because degeneralization also tracks progress through multiple accepting transition sets. We therefore call a decomposable GBA transition progress-consistent if its decomposition produces the same acceptance-progress update as the original transition for every degeneralization index. Only progress-consistent decomposable transitions are eligible for pruning. Let $m_g := |\mathcal{F}_g|$ and $\mathcal{F}_g = \{F_{g,1}, \dots, F_{g,m_g}\}$. For a GBA transition $e^g \in \rightarrow_g$, define its acceptance-set index set as $\text{Acc}(e^g) := \{h \in \{1, \dots, m_g\} \mid e^g \in F_{g,h}\}$. Each state of the degeneralized NBA is represented by (q, ℓ) , where $\ell \in \mathcal{I}_F := \{0, \dots, m_g\}$ is the current acceptance-progress index. Let $\eta(\ell, e^g)$ denote the progress obtained after traversing e^g from progress ℓ . Set $\bar{\ell} = 0$ if $\ell = m_g$ and $\bar{\ell} = \ell$ otherwise. Then $\eta(\ell, e^g) := \max\{r \in \{\bar{\ell}, \dots, m_g\} \mid \{\bar{\ell} + 1, \dots, r\} \subseteq \text{Acc}(e^g)\}$. This update provides a direct test of progress consistency. Given an initial progress ℓ , the candidate decomposition first computes $\eta(\ell, e_{ik}^g)$ and passes the returned index to e_{kj}^g to obtain $\eta(\eta(\ell, e_{ik}^g), e_{kj}^g)$. This result is compared with $\eta(\ell, e_{ij}^g)$, and the decomposition is accepted when the two updates agree for all admissible initial progress indices. The following definition formalizes this criterion.

Definition 7 (Progress-consistent decomposable transition). A decomposable GBA transition $e_{ij}^g \in \rightarrow_g$ is called progress-consistent decomposable if it admits a decomposition (e_{ik}^g, e_{kj}^g) according to Definition 6 such that $\eta(\eta(\ell, e_{ik}^g), e_{kj}^g) = \eta(\ell, e_{ij}^g)$ for every $\ell \in \mathcal{I}_F$. Such a pair (e_{ik}^g, e_{kj}^g) is called a progress-consistent decomposition witness of e_{ij}^g .

Lemma 1. Every NBA transition induced by a progress-consistent decomposable GBA transition is decomposable.

Proof. Let $e_{ij}^g = (q_{g,i}, \sigma_{ij}, q_{g,j})$ be a progress-consistent decomposable GBA transition, and let (e_{ik}^g, e_{kj}^g) be one of its progress-consistent decomposition transitions, where $e_{ik}^g = (q_{g,i}, \sigma_{ik}, q_{g,k})$ and $e_{kj}^g = (q_{g,k}, \sigma_{kj}, q_{g,j})$. Fix an arbitrary acceptance-progress index $\ell \in \mathcal{I}_F$. The transition e_{ij}^g induces the NBA transition $((q_{g,i}, \ell), \sigma_{ij}, (q_{g,j}, \eta(\ell, e_{ij}^g)))$. The transition induces successive NBA transitions through $(q_{g,k}, \eta(\ell, e_{ik}^g))$, with terminal state $(q_{g,j}, \eta(\eta(\ell, e_{ik}^g), e_{kj}^g))$. By progress consistency, $\eta(\eta(\ell, e_{ik}^g), e_{kj}^g) = \eta(\ell, e_{ij}^g)$. Thus, both representations reach the same terminal NBA state. Moreover, $\sigma_{ij} = \sigma_{ik} \cup \sigma_{kj}$ follows from Definition 6. Therefore, the NBA transition induced by e_{ij}^g is decomposable. Since ℓ is arbitrary, the result holds for every NBA transition induced by e_{ij}^g . $\square$

Lemma 2. If a feasible solution can be generated from the original GBA, then an equivalent feasible solution can also be generated from the pruned GBA.

Proof. Let $\mathcal{B}_\varphi$ and $\mathcal{B}_\varphi^-$ be the NBAs generated from the original and pruned GBAs, respectively. Consider a feasible solution Π represented by an accepting run ρ of $\mathcal{B}_\varphi$. A GBA transition removed by infeasible-transition pruning cannot induce a transition used by Π . Otherwise, Π would contain a task requirement that cannot be satisfied by the robot team, which contradicts its feasibility. Now consider a progress-consistent decomposable GBA transition removed during the second pruning step. By Lemma 1, every NBA transition induced by this GBA transition is decomposable. The corresponding decomposition reaches the same terminal NBA state and preserves the acceptance progress. Moreover, its transition labels jointly reproduce the original transition label. Thus, the decomposition introduces no additional task requirements. Applying the decomposition to every affected transition in ρ produces an accepting run of $\mathcal{B}_\varphi^-$. If a transition involved in a decomposition is also pruned, its own decomposition is applied in the pruning order. Since the GBA contains finitely many transitions, this decomposition process terminates with transitions retained in the pruned GBA. The resulting accepting run represents a feasible solution equivalent to Π . $\square$

VI. ON-THE-FLY PLAN GRAPH CONSTRUCTION

After GBA-level pruning, the NBA and the plan graph are constructed synchronously. As the NBA is generated, each candidate transition induces a subtask. The greedy allocation procedure evaluates this subtask from the current plan nodes. A successful allocation extends the plan graph and updates the predicted execution state of the robot team. Once an accepting prefix-suffix structure is reached, an executable plan can be extracted without constructing the complete NBA. To guide this joint construction, a residual-obligation score is used to order newly generated NBA states. In contrast to the original LTL2BA construction, which follows the DFS generation order, the proposed method gives priority to states associated with more promising partial plans. Since the plan graph grows together with the NBA, this ordering also guides the exploration of task-allocation branches and helps obtain a high-quality initial plan earlier.

A. Plan Graph and Plan Node

During NBA construction, feasible task allocations are incrementally recorded in a plan graph. Each plan node associates an NBA state with the corresponding robot assignment and predicted execution state. A plan node is formally defined as follows.

Definition 8 (Plan node). Consider the plan graph $G_P = (V_P, E_P)$, where V_P is the set of plan nodes and $E_P \subseteq V_P \times V_P$ is the set of directed edges. Each plan node is denoted by ν_i , and $V_P = \{\nu_i \mid i = 0, 1, \dots, N_P\}$, where ν_0 is the root node and $N_P + 1$ is the number of nodes in the generated plan graph. Each plan node $\nu_i \in V_P$ is defined as

$$\nu_i = (q_{B,i}, \text{Prog}_i, \omega_i, \mathcal{C}_i, X_i, \Theta_i).$$

Here, $q_{B,i} \in Q_B$ is the NBA state associated with ν_i , and $\text{Prog}_i \in \{\text{pre}, \text{suf}, \text{acc}, \text{dead}\}$ denotes the planning status of ν_i . Specifically, $\text{Prog}_i = \text{pre}$ indicates that the current node is in the prefix stage and $\text{Prog}_i = \text{suf}$ indicates that the current node is in the suffix stage. $\text{Prog}_i = \text{acc}$ indicates that the node completes an accepting prefix-suffix structure, while $\text{Prog}_i = \text{dead}$ indicates that the candidate is discarded and will not be further expanded. For each non-root node ν_i , $\text{par}(\nu_i)$ denotes the predecessor used to compute its stored execution state and to extract the plan. ω_i is the subtask associated with ν_i , and $\mathcal{C}_i$ is the corresponding robot assignment. The vector $X_i = (x_i(r_1), x_i(r_2), \dots, x_i(r_{n_r}))$ denotes the predicted robot positions, where $x_i(r_k)$ is the predicted position of robot r_k at plan node ν_i . The vector $\Theta_i = (\theta_i(r_1), \theta_i(r_2), \dots, \theta_i(r_{n_r}))$ denotes the predicted completion times, where $\theta_i(r_k)$ is the predicted completion time of robot r_k .

By Definition 8, a plan node records the progress in the automaton and the predicted execution state of the robot team. Specifically, $q_{B,i}$, Prog_i , and ω_i describe where the current partial plan is in the NBA and which subtask is being considered. The assignment $\mathcal{C}_i$ specifies which robots execute the current subtask, while X_i and Θ_i summarize the predicted robot configuration after this subtask is completed.

B. Construction of Plan Graph

The plan graph is constructed synchronously with the NBA. The root node is initialized as $\nu_0 = (q_{B,0}, \text{pre}, \emptyset, \emptyset, X_0, \Theta_0)$. For each robot $r_k \in \mathcal{R}$, its initial predicted position and completion time are $x_0(r_k) = x(r_k)$ and $\theta_0(r_k) = 0$, respectively. The same NBA state may be reached through different task-allocation sequences. These sequences may result in different robot assignments and predicted execution states. Therefore, multiple plan nodes may be associated with the same NBA state. For any plan node ν_i , its current plan cost is defined as follows.

$$J(\nu_i) = \max_{r_k \in \mathcal{R}} \theta_i(r_k). \quad (4)$$

When multiple plan nodes are associated with the same NBA state, they are processed in nondecreasing order of this cost.

During construction, $\mathcal{S}_B$ stores the NBA states waiting to be expanded, while Q_B^{exp} contains the states that have already been expanded. For each state removed from $\mathcal{S}_B$, the temporary set $\mathcal{P}_B$ stores the newly discovered successor NBA states. For each feasible extension from a parent plan node ν_i along a candidate transition $e_{B,ij}$, the ordering score $S(\nu_i, e_{B,ij})$ is computed. A lower score gives the corresponding successor NBA state higher priority. If several feasible extensions reach the same successor state $q_{B,j}$, $s_B(q_{B,j})$ retains their smallest score. The score computation is presented in the residual-obligation scoring subsection.

Algorithm 2 summarizes the overall construction procedure. When an NBA state $q_{B,i}$ is expanded, its candidate outgoing transitions are generated. Before planning, these transitions are simplified using the on-the-fly rule of LTL2BA [12]. On-the-fly automaton simplification is one of the main improvements

Algorithm 2 CONSTRUCTPLANGRAPH

Require: Pruned GBA $\mathcal{G}_\varphi^-$, robot team $\mathcal{R}$, and initial plan node ν_0 .

Ensure: Plan graph G_P and initial plan Π_{init} .

```

1:  $G_P \leftarrow (\{\nu_0\}, \emptyset)$ .
2:  $\mathcal{S}_B \leftarrow [q_{B,0}]$  and  $Q_B^{\text{exp}} \leftarrow \emptyset$ .
3:  $\Pi_{\text{init}} \leftarrow \emptyset$ .
4: while  $\mathcal{S}_B \neq \emptyset$  do
5:    $q_{B,i} \leftarrow \text{Pop}(\mathcal{S}_B)$ .
6:    $Q_B^{\text{exp}} \leftarrow Q_B^{\text{exp}} \cup \{q_{B,i}\}$ .
7:    $\mathcal{P}_B \leftarrow \emptyset$ .
8:   Generate the candidate NBA transitions from  $q_{B,i}$ .
9:   Simplify the candidate transitions from  $q_{B,i}$  using the
   on-the-fly transition rule of LTL2BA.
10:  for all remaining candidate transition  $e_{B,ij} : q_{B,i} \xrightarrow{\sigma_{ij}} q_{B,j}$  do
11:    for all plan node  $\nu_i$  associated with  $q_{B,i}$ , in nondecreasing
    order of  $J(\nu_i)$  do
12:       $\nu_j \leftarrow \text{EXTENDPLANGRAPH}(G_P, \nu_i, e_{B,ij}, \mathcal{R})$ .
13:      if  $\nu_j \neq \emptyset$  then
14:        if  $\nu_j.\text{Prog} = \text{acc}$  then
15:          if  $\Pi_{\text{init}} = \emptyset \vee J(\nu_j) < T(\Pi_{\text{init}})$  then
16:            Set  $\Pi_{\text{init}}$  to the plan extracted from  $\nu_j$ .
17:          end if
18:        else
19:          Update  $\mathcal{M}_j$  using the remaining sets of the
          parent nodes of  $\nu_j$ .
20:          Compute  $S(\nu_i, e_{B,ij})$  according to Eq. 8.
21:          if  $q_{B,j} \in Q_B^{\text{exp}}$  then
22:            REUSESUCCESSORS( $G_P, \nu_j, \mathcal{R}$ ).
23:          else if  $q_{B,j} \notin \mathcal{S}_B$  then
24:            if  $q_{B,j} \notin \mathcal{P}_B$  then
25:               $\mathcal{P}_B \leftarrow \mathcal{P}_B \cup \{q_{B,j}\}$ .
26:               $s_B(q_{B,j}) \leftarrow S(\nu_i, e_{B,ij})$ .
27:            else
28:               $s_B(q_{B,j}) \leftarrow \min\{s_B(q_{B,j}), S(\nu_i, e_{B,ij})\}$ .
29:            end if
30:          end if
31:        end if
32:      end if
33:    end for
34:  end for
35:  Sort the states in  $\mathcal{P}_B$  by nondecreasing  $s_B$ , using
  generation order to break ties.
36:  Push them onto  $\mathcal{S}_B$  in reverse order.
37: end while
38: Apply the simplification in Section VI-C to  $G_P$ .
39: return ( $G_P, \Pi_{\text{init}}$ ).

```

of LTL2BA. Each new transition is compared with the transitions already generated from the same NBA state. If two transitions have the same successor and the condition of one implies that of the other, the former is removed immediately. For each remaining transition $e_{B,ij} : q_{B,i} \xrightarrow{\sigma_{ij}} q_{B,j}$, the plan nodes associated with $q_{B,i}$ are processed in nondecreasing

order of their costs. For each parent–transition pair, EXTENDPLANGRAPH first attempts to generate a feasible successor plan node. An accepting successor terminates its branch and provides a candidate plan. For every other feasible successor, its remaining set is updated and its ordering score is computed according to Eq. 8. A feasible successor state that has neither been expanded nor placed in $\mathcal{S}_B$ is added to $\mathcal{P}_B$. If several feasible extensions reach the same successor state, only the smallest score is retained. The states in $\mathcal{P}_B$ are then ordered by their retained scores and added to $\mathcal{S}_B$.

Whenever a returned plan node ν_j satisfies $\nu_j.\text{Prog} = \text{acc}$, its plan is extracted. The extracted plan replaces Π_{init} only if Π_{init} is empty or $J(\nu_j) < T(\Pi_{\text{init}})$. Therefore, Π_{init} stores the lowest-cost accepting plan found during construction. After all pending NBA states have been processed, the plan graph is simplified according to Section VI-C. The algorithm then returns the simplified plan graph G_P and Π_{init} . The EXTENDPLANGRAPH procedure is presented in Algorithm 3.

Algorithm 3 EXTENDPLANGRAPH

Require: Plan graph G_P , parent node ν_i , NBA transition $e_{B,ij} : q_{B,i} \xrightarrow{\sigma_{ij}} q_{B,j}$, and robot team $\mathcal{R}$.

Ensure: Successor plan node ν_j , or $\emptyset$ if the assignment fails.

```

1: Obtain the induced subtask  $\omega_{ij} = (\sigma_{ij}, \sigma_i^s)$ .
2: Compute the successor progress flag  $\text{Prog}_j$ , setting
    $\text{Prog}_j \leftarrow \text{acc}$  if the extension completes an accepting
   prefix–suffix structure.
3:  $\nu_h \leftarrow$  a plan node associated with  $q_{B,j}$  whose progress
   flag is  $\text{Prog}_j$  and whose subtask is  $\omega_{ij}$ , if one exists.
4:  $(\hat{\mathcal{C}}, \hat{X}, \hat{\Theta}, \hat{J}) \leftarrow \text{GREEDYASSIGN}(\omega_{ij}, \nu_i, \mathcal{R})$ .
5: if GREEDYASSIGN fails then
6:   return  $\emptyset$ .
7: end if
8: if  $\nu_h$  exists then
9:   Add  $(\nu_i, \nu_h)$  to  $E_P$ .
10:  if  $\hat{J} < J(\nu_h)$  then
11:    Set  $\text{par}(\nu_h) \leftarrow \nu_i$ ,  $\mathcal{C}_h \leftarrow \hat{\mathcal{C}}$ ,  $X_h \leftarrow \hat{X}$ , and  $\Theta_h \leftarrow \hat{\Theta}$ ;
    propagate the improvement to descendants.
12:  end if
13:   $\nu_j \leftarrow \nu_h$ .
14: else
15:   Create  $\nu_j = (q_{B,j}, \text{Prog}_j, \omega_{ij}, \hat{\mathcal{C}}, \hat{X}, \hat{\Theta})$ .
16:   Set  $\text{par}(\nu_j) \leftarrow \nu_i$ , insert  $\nu_j$  into  $V_P$ , and add  $(\nu_i, \nu_j)$ 
   to  $E_P$ .
17: end if
18: if  $\nu_j.\text{Prog} = \text{acc}$  then
19:   Record  $\nu_j$  as a feasible solution leaf.
20: end if
21: return  $\nu_j$ .

```

1) *ExtendPlanGraph*: Algorithm 3 extends the plan graph from a parent plan node ν_i along an NBA transition $e_{B,ij}$. It first constructs the induced subtask $\omega_{ij} = (\sigma_{ij}, \sigma_i^s)$ and computes the successor progress flag Prog_j . The flag records whether the extension remains in the prefix stage, enters the suffix stage, or completes an accepting prefix–suffix structure. An existing plan node ν_h is compatible with this extension if

it is associated with $q_{B,j}$ and satisfies $\text{Prog}_h = \text{Prog}_j$ and $\omega_h = \omega_{ij}$.

The procedure next calls function **GREEDYASSIGN**($\omega_{ij}, \nu_i, \mathcal{R}$) to evaluate the candidate extension. For each required action proposition $a_\ell \in \mathcal{A}(\omega_{ij})$, the execution requirement $c(a_\ell) = (\tau_\ell, n_\ell, l_\ell)$ specifies the required robot type, number of robots, and target region. For every robot $r_k \in \mathcal{R}_{\tau_\ell}$, its predicted arrival time from ν_i is computed as follows.

$$\hat{t}_i(r_k, a_\ell) = \theta_i(r_k) + \frac{d_k(x_i(r_k), l_\ell)}{v(r_k)}. \quad (5)$$

Here, $x_i(r_k)$ and $\theta_i(r_k)$ are stored in ν_i . A robot is eligible for a_ℓ if its predicted arrival time is finite and it has not been assigned to another action proposition in the same subtask.

The action propositions in $\mathcal{A}(\omega_{ij})$ are processed in increasing order of their indices. For each a_ℓ , the n_ℓ eligible robots with the smallest predicted arrival times are selected and stored in $\hat{\mathcal{C}}(a_\ell)$. The predicted positions of the selected robots are updated to the corresponding target regions, and their predicted completion times are set to the latest predicted arrival time among all robots assigned to ω_{ij} . Robots not assigned to ω_{ij} retain their states from X_i and Θ_i .

These updates produce $\hat{X} = (\hat{x}(r_1), \dots, \hat{x}(r_{n_r}))$ and $\hat{\Theta} = (\hat{\theta}(r_1), \dots, \hat{\theta}(r_{n_r}))$. The resulting node cost is $\hat{J} = \max_{r_k \in \mathcal{R}} \hat{\theta}(r_k)$. If fewer than n_ℓ eligible robots are available for any required action proposition, **GREEDYASSIGN** fails and the candidate extension is discarded.

If **GREEDYASSIGN** succeeds and a compatible plan node ν_h exists, **EXTENDPLANGRAPH** adds (ν_i, ν_h) to E_P . If $\hat{J} < J(\nu_h)$, the representative predecessor of ν_h is updated to ν_i . Its robot assignment, predicted positions, and predicted completion times are replaced by $\hat{\mathcal{C}}$, $\hat{X}$, and $\hat{\Theta}$, respectively. The resulting improvement is then propagated to its successor nodes. If no compatible plan node exists, a new node $\nu_j = (q_{B,j}, \text{Prog}_j, \omega_{ij}, \hat{\mathcal{C}}, \hat{X}, \hat{\Theta})$ is created. Its representative predecessor is set to ν_i . The new node is inserted into V_P , and the edge (ν_i, ν_j) is added to E_P .

2) *Reuse of Existing Successors*: When a plan node ν_j reaches an NBA state $q_{B,j}$ that has already been expanded, its outgoing NBA transitions and successor structure are already available. Therefore, these transitions are not generated again. The **REUSESUCCESSORS** procedure connects ν_j to the existing successor plan nodes and determines whether their representative execution states should be updated.

Algorithm 4 REUSESUCCESSORS

Require: Plan graph G_P , plan node ν_j associated with $q_{B,j} \in Q_B^{\text{exp}}$, and robot team $\mathcal{R}$.

Ensure: Updated plan graph G_P .

```

1: for all  $(\nu_p, \nu_h) \in E_P : q_{B,p} = q_{B,j}, \text{Prog}_p = \text{Prog}_j, (\nu_j, \nu_h) \notin E_P$  do
2:   Add  $(\nu_j, \nu_h)$  to  $E_P$ .
3:   if  $J(\nu_j) < \min\{J(\nu_\ell) \mid (\nu_\ell, \nu_h) \in E_P, \nu_\ell \neq \nu_j\}$  then
4:     Set  $\text{par}(\nu_h) \leftarrow \nu_j$ .
5:     Recompute  $\mathcal{C}_h, X_h$ , and  $\Theta_h$  from  $\nu_j$ .
6:     Propagate the update to the descendants of  $\nu_h$ .
7:   end if
8: end for
```

For each existing successor ν_h , the new edge is retained. If ν_j has the smallest cost among the parent nodes of ν_h , it is selected as the representative predecessor. The robot assignment and predicted execution state of ν_h are then recomputed from ν_j . Its cost is updated accordingly, and the updated state is propagated to its descendants. Otherwise, the representative predecessor and the stored execution state of ν_h remain unchanged. This cost-based update follows the planning-decision-tree rule of Chen et al. [11], which was validated experimentally.

3) *Residual-Obligation Scoring of NBA States*: After a feasible nonaccepting successor plan node has been generated, Algorithm 2 updates its remaining set and computes the ordering score $S(\nu_i, e_{B,ij})$. The score is used to order the successor NBA state $q_{B,j}$ in the current pending set $\mathcal{P}_B$. It combines the current plan cost $J(\nu_i)$ with an estimate of the positive action requirements that remain.

This estimate is based on a must-eventually action set extracted once from the abstract syntax tree $\mathcal{T}_\varphi$. For each subformula ψ , $\text{ME}(\psi) \subseteq \mathcal{A}$ denotes a set of positive action propositions that must each be executed at least once during the subsequent execution for ψ to be satisfied. This set is obtained using the recursive rules below. The remaining set associated with each plan node ν_i is denoted by $\mathcal{M}_i$. At the root node ν_0 , it is initialized as $\mathcal{M}_0 = \text{ME}(\varphi)$. In AST, $\Diamond\psi$ and $\Box\psi$ are represented in $\mathcal{T}_\varphi$ as $\top \mathcal{U} \psi$ and $\perp \mathcal{V} \psi$, respectively.

- 1) The constants $\top$ and $\perp$ do not introduce any positive action proposition that must be completed. That is, if $\psi = \top$ or $\psi = \perp$, then $\text{ME}(\psi) = \emptyset$.
- 2) If $\psi = a$ with $a \in \mathcal{A}$, then $\text{ME}(\psi) = \{a\}$. If $\psi = p$ with $p \in \mathcal{P}$, or if $\psi = \neg\pi$ with $\pi \in \mathcal{AP}$, then $\text{ME}(\psi) = \emptyset$. That is, only positive action propositions are included.
- 3) If $\psi = \psi_1 \wedge \psi_2$, then $\text{ME}(\psi) = \text{ME}(\psi_1) \cup \text{ME}(\psi_2)$. Since both subformulas must be satisfied, the action propositions are collected from both sides.
- 4) If $\psi = \psi_1 \vee \psi_2$, then $\text{ME}(\psi) = \text{ME}(\psi_1) \cap \text{ME}(\psi_2)$. Since either branch may be selected, only action propositions that are unavoidable in both branches are kept.
- 5) If $\psi = \bigcirc\psi_1$, then $\text{ME}(\psi) = \text{ME}(\psi_1)$. The next operator shifts the satisfaction time but does not change the unavoidable positive action propositions.
- 6) If $\psi = \psi_1 \mathcal{U} \psi_2$ or $\psi = \psi_1 \mathcal{V} \psi_2$, then $\text{ME}(\psi) = \text{ME}(\psi_2)$, where $\mathcal{V}$ denotes the release operator. In both cases, the positive action propositions retained from the

right operand must be satisfied during the subsequent execution.

The set $\mathcal{M}_i$ is a formula-level summary of unavoidable positive action propositions rather than a set that must be satisfied immediately at the node ν_i . It is used to estimate the remaining effort before all future subtasks have been generated.

For a feasible successor plan node ν_j , its remaining set combines those of its parent nodes and then removes the positive action propositions executed by the induced subtask. It is updated as follows.

$$\mathcal{M}_j = \left(\bigcup_{(\nu_i, \nu_j) \in E_P} \mathcal{M}_i \right) \setminus \mathcal{A}(\omega_{ij}). \quad (6)$$

For each $a \in \mathcal{M}_j$ with $c(a) = (\tau_a, n_a, l_a)$, we compute the arrival-time estimates of all robots in $\mathcal{R}_{\tau_a}$ using the predicted positions X_i and completion times Θ_i stored in ν_i , and take the n_a -th smallest value, denoted by $h_a(\nu_i)$. The residual pressure of $(\nu_i, e_{B,ij})$ is

$$H(\nu_i, e_{B,ij}) = \begin{cases} \max_{a \in \mathcal{M}_j} h_a(\nu_i), & \mathcal{M}_j \neq \emptyset, \\ J(\nu_i), & \mathcal{M}_j = \emptyset. \end{cases} \quad (7)$$

The candidate score is

$$S(\nu_i, e_{B,ij}) = \max\{J(\nu_i), H(\nu_i, e_{B,ij})\}. \quad (8)$$

If the same successor NBA state is discovered from multiple active plan nodes in the current pending batch, the smallest candidate score is retained for ordering that successor in the batch.

C. Plan-Graph Simplification

After the plan graph has been constructed, nodes and edges that cannot contribute to an accepting plan are removed through two simplification steps. First, the simplification retains only the plan nodes that are both reachable from the root node ν_0 and capable of reaching a plan node with $\text{Prog} = \text{acc}$. These nodes are identified by combining forward reachability from ν_0 along the outgoing edges in E_P with backward reachability from all nodes with $\text{Prog} = \text{acc}$ along the incoming edges. All other plan nodes are removed together with their incident edges. An edge is retained only if both of its endpoint nodes are retained.

Second, a plan node is regarded as a leaf if it has no outgoing edge in E_P . Every leaf node ν_i satisfying $\nu_i.\text{Prog} \neq \text{acc}$ is removed together with its incoming edges. This removal may turn its parent nodes into new nonaccepting leaves. The same rule is therefore applied repeatedly until every remaining leaf satisfies $\nu_i.\text{Prog} = \text{acc}$. Accepting leaves are retained because they represent completed prefix-suffix plans.

Lemma 3. Plan-graph simplification preserves every feasible accepting plan represented in G_P .

Proof. Every feasible accepting plan in G_P corresponds to a path from the root node ν_0 to a plan node with $\text{Prog} = \text{acc}$. Every node on this path is reachable from ν_0 and can reach the accepting node. Moreover, the final node is accepting, while

every preceding node has an outgoing edge along the path. Therefore, the removal of nonaccepting leaf nodes does not affect this path. The feasible accepting plan is thus preserved after simplification. $\square$

D. Plan Extraction

Whenever an accepting prefix-suffix structure is reached during construction, the corresponding solution leaf is recorded. Because plan nodes can be reused, a plan node may have multiple parent nodes in G_P . The executable plan, however, is extracted through the representative-predecessor chain. Starting from a recorded solution leaf ν_f , the algorithm repeatedly follows $\text{par}(\nu)$ until the root node is reached, and then reverses the obtained sequence to form a root-to-leaf plan.

For each node on this representative path, the stored assignment map, predicted robot positions, and completion times define the executable subtask transition. Thus, the extracted plan is consistent with the robot assignments and predicted execution states stored in the plan graph. The set of parent nodes is used for node reuse and graph simplification, but it is not enumerated during plan extraction.

VII. BRANCH-AND-BOUND OPTIMIZATION

After the plan graph has been constructed and simplified, the greedy allocation used during its construction provides an initial feasible plan, which is not necessarily optimal. We therefore apply a warm-started branch-and-bound (BnB) search directly to G_P . The search jointly selects a path from the root node ν_0 to a plan node with $\text{Prog} = \text{acc}$ and feasible robot assignments for the subtasks along that path. Such a path determines a finite plan Π_{fin} associated with an accepting prefix-suffix run. The objective is to minimize J_φ defined in Eq. (3). The initial feasible plan initializes the incumbent plan Π^* and its cost J^* . When all candidates that can improve the incumbent have been examined or pruned, the returned plan has the minimum makespan over the paths and assignments retained in G_P . If the search is stopped by a runtime or memory limit, the best incumbent found so far is returned.

A. Node Expansion

Each BnB search node is represented by

$$\mu_i = (\nu_i, X_i^b, \Theta_i^b, \mathcal{C}_i^b),$$

where ν_i is the associated plan node. The vectors X_i^b and Θ_i^b denote the predicted robot positions and completion times carried by μ_i , respectively. The robot assignment $\mathcal{C}_i^b$ specifies the robots assigned to the required action propositions in ω_i . For the root BnB node, $\mathcal{C}_0^b = \emptyset$.

The expansion of a BnB node follows the edges of the plan graph. Consider a node $\mu_i = (\nu_i, X_i^b, \Theta_i^b, \mathcal{C}_i^b)$. For each edge $(\nu_i, \nu_j) \in E_P$, the search considers every feasible robot assignment $\mathcal{C}_j^b$ for the subtask ω_j . A feasible assignment satisfies the robot-type, cardinality, disjointness, reachability, and holding-condition constraints. For each feasible $\mathcal{C}_j^b$, the state-update rules in Sec. VI-B1 are applied to X_i^b and Θ_i^b , producing X_j^b and Θ_j^b . The resulting child node is $\mu_j =$

Algorithm 5 Warm-Started BnB on the Generated Plan Graph

Require: Plan graph $G_P = (V_P, E_P)$, incumbent plan Π^* , incumbent cost J^* .

Ensure: Improved plan Π^* and cost J^* .

```

1: Set  $X_0^b \leftarrow X_0$ ,  $\Theta_0^b \leftarrow \Theta_0$ , and  $\mathcal{C}_0^b \leftarrow \emptyset$ .
2: Initialize the root BnB node  $\mu_0 = (\nu_0, X_0^b, \Theta_0^b, \mathcal{C}_0^b)$ .
3: Insert  $\mu_0$  into a priority queue  $\mathcal{Q}$  with key  $LB(\mu_0)$ .
4: while  $\mathcal{Q} \neq \emptyset$  and the stopping limit is not reached do
5:   Pop  $\mu_i = (\nu_i, X_i^b, \Theta_i^b, \mathcal{C}_i^b)$  with the smallest lower bound from  $\mathcal{Q}$ .
6:   if  $LB(\mu_i) \geq J^*$  then
7:     break
8:   end if
9:   if  $\nu_i.\text{Prog} = \text{acc}$  then
10:    if  $J(\mu_i) < J^*$  then
11:      Recover the plan through the BnB predecessor chain and update  $(\Pi^*, J^*)$ .
12:    end if
13:    continue
14:  end if
15:  for all  $\nu_j$  such that  $(\nu_i, \nu_j) \in E_P$  do
16:    for all feasible robot assignments  $\mathcal{C}_j^b$  for  $\omega_j$  do
17:      Propagate  $X_i^b$  and  $\Theta_i^b$  to obtain  $X_j^b$  and  $\Theta_j^b$ .
18:      Create  $\mu_j = (\nu_j, X_j^b, \Theta_j^b, \mathcal{C}_j^b)$  and store  $\mu_i$  as its BnB predecessor.
19:      if  $LB(\mu_j) < J^*$  then
20:        Insert  $\mu_j$  into  $\mathcal{Q}$  with key  $LB(\mu_j)$ .
21:        Use a greedy rollout from  $\mu_j$  to update  $(\Pi^*, J^*)$  if a better feasible plan is found.
22:      end if
23:    end for
24:  end for
25: end while
26: return  $(\Pi^*, J^*)$ .

```

$(\nu_j, X_j^b, \Theta_j^b, \mathcal{C}_j^b)$. Robots not assigned to ω_j retain their states from X_i^b and Θ_i^b . Each child stores μ_i as its predecessor in the BnB search tree, allowing the complete assignment sequence to be recovered when a plan node with $\text{Prog} = \text{acc}$ is reached.

B. Lower Bound

For a plan node ν_i , let $\mathcal{A}_{\text{must}}(\nu_i) \subseteq \mathcal{A}$ denote the set of positive action propositions that occur in every accepting path from ν_i . For an accepting terminal node, $\mathcal{A}_{\text{must}}(\nu_i) = \emptyset$. For every other plan node, the set is computed backward as follows.

$$\mathcal{A}_{\text{must}}(\nu_i) = \bigcap_{(\nu_i, \nu_j) \in E_P} (\mathcal{A}(\omega_j) \cup \mathcal{A}_{\text{must}}(\nu_j)). \quad (9)$$

Here, ω_j is the subtask stored in the successor plan node ν_j . Thus, an action is retained only if it is required through every accepting continuation from ν_i . The sets $\mathcal{A}_{\text{must}}(\nu_i)$ are computed once from the simplified plan graph before the BnB search and reused throughout the search.

For a BnB node μ_i , its current cost $J(\mu_i)$ is evaluated from its execution state using the same makespan definition as $J(\nu_i)$

in Eq. (4). For each $a \in \mathcal{A}_{\text{must}}(\nu_i)$ with $c(a) = (\tau_a, n_a, l_a)$, the arrival times of the compatible robots are computed from the execution state carried by μ_i , according to Eq. (5). As in the residual-obligation score, $h_a(\mu_i)$ denotes the n_a -th smallest finite arrival time. If fewer than n_a compatible robots can reach l_a , then $h_a(\mu_i) = +\infty$. The lower bound is defined as below.

$$LB(\mu_i) = \max \left\{ J(\mu_i), \max_{a \in \mathcal{A}_{\text{must}}(\nu_i)} h_a(\mu_i) \right\}. \quad (10)$$

If $\mathcal{A}_{\text{must}}(\nu_i) = \emptyset$, the second term is omitted and $LB(\mu_i) = J(\mu_i)$.

C. Upper Bound

After a child BnB node is generated, a greedy rollout is performed from that node to construct a feasible completion and possibly improve the incumbent. At each rollout step, the next plan node is selected using the robot demand of its immediate subtask and the actions that remain mandatory.

Resource-aware selection rules have been used in branch-and-bound methods for resource-constrained project scheduling [24]. Priority-rule heuristics have also been used to construct feasible upper-bound solutions efficiently [25]. Following this idea, we compare the robot demand of each immediate subtask with the mandatory demand of its continuation.

Consider a current BnB node μ_i and a successor plan node ν_j satisfying $(\nu_i, \nu_j) \in E_P$. The normalized robot demand of its immediate subtask is defined as

$$D(\omega_j) = \sum_{a \in \mathcal{A}(\omega_j)} \frac{n_a}{|\mathcal{R}_{\tau_a}|}. \quad (11)$$

The total normalized demand associated with selecting ν_j is defined as

$$W(\nu_j) = \sum_{a \in \mathcal{A}(\omega_j) \cup \mathcal{A}_{\text{must}}(\nu_j)} \frac{n_a}{|\mathcal{R}_{\tau_a}|}. \quad (12)$$

The priority key of ν_j is $K(\nu_j) = (W(\nu_j), -D(\omega_j))$. The successor with the lexicographically smallest $K(\nu_j)$ is selected. Dividing n_a by $|\mathcal{R}_{\tau_a}|$ measures the proportion of the available robots required by a , allowing requirements for different robot types to be compared. The first component favors a continuation with less mandatory robot demand. If several successors have the same value of $W(\nu_j)$, the second component gives priority to the immediate subtask with the greater normalized demand $D(\omega_j)$. Any remaining tie is resolved by generation order.

A greedy robot assignment is computed only for the selected successor. If the assignment succeeds, the resulting child BnB node becomes the current node and the procedure is repeated. If it fails, the rollout terminates without updating the incumbent. When a node with $\text{Prog} = \text{acc}$ is reached, the resulting feasible plan updates (Π^*, J^*) if its cost is smaller than J^* . This priority rule guides only the rollout and is not used for BnB pruning.

D. Branching and Pruning

The priority queue $\mathcal{Q}$ stores the BnB nodes that have been generated but not yet expanded. These nodes are ordered by nondecreasing values of LB . At each iteration, the node with the smallest lower bound is removed from $\mathcal{Q}$ and expanded. A node μ_i is pruned if $LB(\mu_i) \geq J^*$, because it cannot improve the incumbent plan. If no feasible robot assignment exists for a successor subtask, the corresponding child node is not generated.

The search terminates when $\mathcal{Q}$ is empty or when its smallest lower bound is no smaller than J^* . In either case, the incumbent has the minimum cost over the paths and robot assignments retained in G_P . If a prescribed runtime or memory limit is reached before either condition holds, the search returns the best incumbent found so far.

VIII. PLAN ADAPTATION UNDER AGENT FAILURES

During task execution, a robot may become unavailable because of an unexpected failure and can no longer complete its assigned subtasks. The unfinished part of the mission must therefore be replanned using the remaining robot team.

The unavailable robots are first removed from the robot team. The current execution progress and the states of the remaining robots then define a new root for replanning. The replanning search is initialized without a warm start because the previous assignment may involve the unavailable robot. Starting from the new root, the branch-and-bound procedure reassigns the unfinished subtasks over the retained plan graph while enforcing the original temporal and collaboration constraints. During the search, the upper-bound procedure uses a greedy rollout to construct a feasible completion. This allows the planner to obtain the first adapted solution quickly. The solution then serves as the incumbent for subsequent improvement. The final assignment replaces the unexecuted part of the original plan.

IX. COMPLEXITY ANALYSIS

This section analyzes the computational cost of the proposed framework in three stages: GBA-level pruning, on-the-fly plan-graph construction, and BnB optimization over the generated plan graph.

Let $n_G = |\mathcal{Q}_g|$, let d_G be the maximum GBA out-degree, and recall that $m_g = |\mathcal{F}_g|$. The feasibility screening is linear in the total number of action requirements contained in the candidate GBA transitions. After the progress updates are computed for all GBA transitions, the progress-consistent decomposable-transition pruning has complexity $O(|\rightarrow_g| m_g + n_G d_G^3 (|\mathcal{AP}| + m_g))$.

The cost of constructing the plan graph mainly comes from generating and updating plan nodes. For each parent plan node ν_i and induced subtask ω_{ij} , greedy allocation requires $O(\sum_{a \in \mathcal{A}(\omega_{ij})} |\mathcal{R}_{\tau_a}| \log |\mathcal{R}_{\tau_a}| + n_r)$ time when the eligible robots are sorted by predicted arrival time and travel distances are cached. The score uses the same arrival-time calculation for the actions in $\mathcal{M}_j$. After a feasible assignment is obtained, a direct search for an existing plan node costs $O(|V_P|)$, and ordering the corresponding successor states costs

$O(|\mathcal{P}_B| \log |\mathcal{P}_B|)$. Existing successor plan nodes are reused whenever possible. If a stored execution state is improved, the update is passed to its descendants, with a cost determined by the affected edges. Finally, plan-graph simplification costs $O(|V_P| + |E_P|)$.

The BnB search follows the edges in E_P and considers all feasible robot assignments for each subtask. For an edge $(\nu_i, \nu_j) \in E_P$, at most $\prod_{a \in \mathcal{A}(\omega_j)} \binom{|\mathcal{R}_{\tau_a}|}{n_a}$ assignments are considered. Because these choices combine at successive plan nodes, the search time is exponential in the worst case. All sets $\mathcal{A}_{\text{must}}(\nu_i)$ are computed once in $O(|\mathcal{A}|(|V_P| + |E_P|))$ time and then reused. The rollout selects an assignment greedily at each step and does not enumerate all possible combinations. Each insertion into or removal from $\mathcal{Q}$ costs $O(\log |\mathcal{Q}|)$.

Task complexity can cause the generated automaton to grow exponentially with $|\varphi|$. For the resulting GBA, the pruning cost is linear in n_G and cubic in d_G . With a fixed subtask structure, greedy allocation costs approximately $O(n_r \log n_r)$ per extension, whereas the BnB branching factor is bounded by $\prod_i \binom{r_i}{k_i}$. Although these worst-case trends remain, GBA-level pruning reduces the transitions propagated to planning, score-guided joint construction obtains a feasible plan early, and plan-graph reuse avoids repeated successor generation. The initial plan also provides an incumbent for pruning the subsequent BnB search.

X. THEORETICAL ANALYSIS

Lemma 4. The on-the-fly plan-graph construction is sound and complete, i.e., if a feasible solution exists, the initial algorithm is able to find a solution and the solution satisfies φ .

Proof. Suppose that a feasible plan exists. By Lemma 2, an equivalent feasible accepting run is preserved after GBA pruning. During the joint construction of the NBA and the plan graph, every transition along the corresponding finite prefix-suffix run is eventually considered. Since the run represents a feasible plan, the subtask induced by each transition admits a valid robot assignment. The construction therefore creates a successor plan node or connects the current node to an existing successor structure. Repeating this argument along the run produces a plan node with $\text{Prog} = \text{acc}$. By Lemma 3, the corresponding path is preserved after plan-graph simplification. Hence, the construction returns a plan whenever a feasible one exists.

Conversely, every returned plan is extracted from a plan node ν_f satisfying $\nu_f.\text{Prog} = \text{acc}$. Tracing the representative predecessors from ν_f to ν_0 produces a sequence of feasible plan-graph extensions. Each extension corresponds to an NBA transition and contains a valid robot assignment for its induced subtask. The resulting NBA-state sequence forms an accepting prefix-suffix run. Its induced word is therefore accepted by $\mathcal{B}_\varphi$ and satisfies φ . Thus, every plan returned by the construction is feasible and satisfies φ . $\square$

Theorem 1. *The proposed algorithm is sound and complete.*

Proof. By Lemma 2, GBA pruning preserves the existence of a feasible solution. Lemma 4 further shows that the task allocation obtained from the plan graph induces a feasible plan

satisfying φ . During BnB optimization, every child follows an edge in E_P and is generated using a feasible robot assignment for the corresponding subtask. The rollout follows the same plan-graph structure and constructs a feasible assignment at every step. Moreover, the incumbent is updated only after reaching a plan node with $\text{Prog} = \text{acc}$. Hence, every updated incumbent corresponds to an accepting prefix-suffix run and satisfies φ . If no improved plan is found, the initial feasible incumbent is retained. Therefore, the complete algorithm returns a feasible plan whenever one exists, and every plan it returns satisfies φ . $\square$

XI. NUMERICAL EXPERIMENTS

In this section, we present five numerical experiments, implemented in Cython 0.29.24 on a workstation with a 3.40-GHz Intel Core i7-11390H CPU and 16 GB RAM. The numerical experiments use the $40\text{ m} \times 40\text{ m}$ warehouse setting introduced in Example 1 and shown in Fig. 1, where heterogeneous robots perform region-level operations under temporal and collaboration constraints. Obstacles are randomly generated within the workspace. First, we analyze the BnB algorithm using a numerical example, focusing on the evolution of the search bounds, the pruning ratio, and the improvement in solution cost. Second, we conduct an ablation study with two variants to examine the effects of GBA-level pruning and task allocation during NBA generation on the time to the first feasible plan. Third, we compare the first feasible plan with the best plan found in the generated plan graph for four LTL specifications to evaluate the quality of the initial solution. Fourth, we compare our method with the poset- and MILP-based methods in [3], [7] under different LTL specifications and robot-team sizes to evaluate scalability. Finally, we introduce robot-unavailability events during task execution to examine whether the unfinished task can be rapidly reassigned to the remaining robot team.

A. Branch-and-Bound Optimization

This experiment analyzes the BnB search process by examining the evolution of the search bounds, the pruning ratio, and the improvement in solution cost. Consider a warehouse cargo-handling and safety-inspection task specified by $\varphi = \Diamond a_1 \wedge \Diamond a_2 \wedge \Diamond(a_3 \wedge \Diamond(a_6 \wedge \Diamond(a_4 \vee a_5))) \wedge \Diamond a_7 \wedge \Diamond a_9 \wedge \Diamond a_{10}$. Specifically, the team consists of five robot types, with five robots of each type, and the action requirements are $c(a_1) = (1, 3, l_1)$, $c(a_2) = (2, 4, l_2)$, $c(a_3) = (2, 1, l_3)$, $c(a_4) = (1, 2, l_4)$, $c(a_5) = (5, 4, l_5)$, $c(a_6) = (3, 3, l_6)$, $c(a_7) = (1, 3, l_7)$, $c(a_8) = (1, 2, l_8)$, $c(a_9) = (3, 3, l_8)$, and $c(a_{10}) = (4, 1, l_8)$. In this task, type-1 robots handle incoming cargo at l_1 and transfer it to l_4 , while type-2 robots perform temporary storage and order picking. One type-2 robot picks an order at l_3 , after which two type-3 robots service the equipment at l_6 . The cargo is then either loaded at l_4 by two type-1 robots or inspected at l_5 by three type-5 robots. In addition, three type-3 robots inspect the dispatch equipment at l_8 , while one type-4 robot monitors the dispatch operation. Robot initial positions are randomly generated in the workspace.

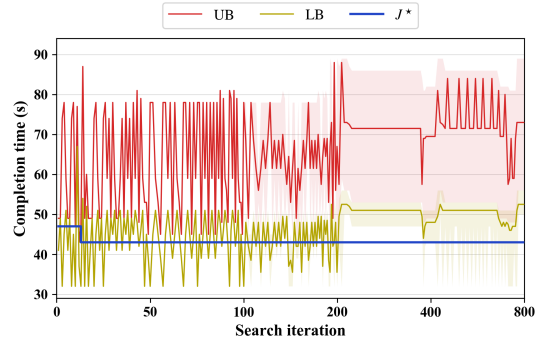


Fig. 2. Evolution of the BnB bounds over the first 800 iterations. UB and LB are the node bounds, and J^* is the incumbent completion time. Iterations 1–100 are shown individually; the remaining intervals are grouped in windows of 2, 4, and 8. Solid curves show the medians, and shaded regions show the minimum–maximum ranges.

As shown in Fig. 2, the initial plan obtained from the plan graph provides an incumbent with $J^* = 47.21\text{s}$ before the BnB search begins. The first pruning occurs at 0.2910s because this initial incumbent allows nodes with $\text{LB} \geq J^*$ to be discarded immediately. The remaining nodes are selected in nondecreasing order of LB. Each feasible completion provides an upper bound UB, and J^* is updated whenever a smaller completion time is found. The best solution is obtained at 2.3679s , reducing the incumbent to $J^* = 43.32\text{s}$. Overall, the BnB search prunes 122,362 of the 138,145 visited assignment nodes, corresponding to a pruning ratio of 88.57%. These results demonstrate the effectiveness of search and pruning in reducing the search effort.

B. Ablation of GBA-Level Pruning and On-the-Fly Plan Graph Construction

This experiment evaluates GBA-level pruning and on-the-fly plan graph construction for tasks of increasing complexity. We use the same robot team as in Sec. XI-A. Consider the patrol task specified as $\varphi = \bigwedge_{i=1}^m \Diamond a_i$, where m is the number of required patrol actions and determines the task complexity. For each tested value of m , the specification contains the first m action propositions. For $i = 1, \dots, 8$, $c(a_i) = (1, 3, l_i)$, so each action requires three type-1 robots to patrol region l_i . For $i = 9, \dots, 15$, $c(a_i) = (2, 3, l_{i-8})$, so three type-2 robots are required to patrol the corresponding region l_{i-8} . In each test, the robot initial positions are randomly generated in the workspace and then kept identical across all configurations. Each configuration is run 25 times, and the results are averaged over these runs.

The proposed method is compared with two cases. Case I retains GBA-level pruning but postpones plan-graph construction until the complete NBA has been generated. Case II generates the complete NBA before performing both pruning and plan-graph construction.

Table I shows that the proposed method obtains the first feasible plan faster than Case I for all tested tasks. Specifically, as task complexity increases, our method requires about one-third less time than Case I to obtain the first feasible plan. This reduction arises because task allocation proceeds together with

TABLE I
EFFECT OF ON-THE-FLY PLAN GRAPH CONSTRUCTION

m	Proposed	Case I
	T_{first} (s)	T_{first} (s)
12	10.4065	12.7062
13	30.7849	53.2725
14	156.0395	217.4069
15	723.6905	1037.2168

NBA generation, allowing a plan to be returned as soon as an accepting prefix-suffix structure is reached. In contrast, Case I incurs the cost of constructing the complete NBA before task allocation begins.

Table II compares the proposed method with Case II, where the NBA size is reported as $(|Q_B|, |\rightarrow_B|)$. For the tested tasks, Case II constructs many more NBA transitions before pruning, although its pruned NBA has the same size as that generated by the proposed method. Since the proposed pruning is performed before the GBA-to-NBA transformation, the removed GBA transitions are neither converted into NBA transitions nor stored in the NBA. This reduces both the computation required for subsequent NBA construction and the memory used to store its transition relation. In contrast, Case II constructs and stores the complete transition structure before removing unnecessary transitions. The shorter first-solution time of the proposed method therefore results from this reduction in NBA construction together with task allocation during NBA generation.

Overall, although the proposed method does not change the worst-case exponential growth of the NBA, it substantially mitigates its practical impact. GBA-level pruning prevents unnecessary transitions from entering the NBA, while on-the-fly plan graph construction uses the generated NBA transitions for task allocation immediately. The proposed method therefore reduces automaton-construction and obtains the first feasible plan without waiting for the complete NBA, which is particularly beneficial for complex tasks.

C. First-Solution Quality

In this experiment, we examine the quality of the first feasible plan returned during on-the-fly plan graph construction. We consider various LTL specifications and compare the first feasible plan with both the best plan found in the completed plan graph and the global optimum. We use the same workspace and robot team as in Sec. XI-A. The considered specifications are as follows:

$$\begin{aligned}
\varphi_1 &= \Diamond(a_2 \wedge \Diamond a_1) \wedge \Diamond(a_3 \wedge \Diamond(a_5 \wedge \neg p_5) \\
&\quad \wedge \Diamond a_4 \wedge \Diamond a_6 \wedge \Diamond a_7 \wedge \Diamond a_8), \\
\varphi_2 &= \Diamond a_2 \wedge \Diamond a_1 \wedge \Diamond a_3 \wedge \Diamond(a_3 \vee a_4) \\
&\quad \wedge \Diamond a_7 \wedge \Box \Diamond a_5, \\
\varphi_3 &= \Diamond a_9 \wedge \Diamond a_{10} \wedge \Diamond(a_3 \wedge a_4) \wedge \Diamond a_5 \\
&\quad \wedge \Box \neg p_1 \wedge \Diamond(a_7 \wedge \Diamond a_8) \\
&\quad \wedge (\neg a_5 \mathcal{U} (a_3 \wedge a_4)), \\
\varphi_4 &= \Diamond a_9 \wedge \Diamond a_{10} \wedge \Diamond a_{11} \wedge (\neg a_1 \mathcal{U} a_2) \\
&\quad \wedge \Diamond a_2 \wedge \Diamond a_1 \wedge \Diamond a_5 \wedge \Diamond a_4 \\
&\quad \wedge (\Box a_4 \mathcal{U} a_5).
\end{aligned}$$

These specifications involve various temporal operators and capture complex requirements commonly encountered in warehouse cargo-handling and safety-inspection tasks. In particular, φ_1 describes a multistage cargo-handling task and a safety check. Specification φ_2 represents a persistent warehouse inspection task that includes an alternative operation and recurring patrol. Specification φ_3 captures collaborative equipment maintenance subject to task-precedence and restricted-region constraints. Specification φ_4 , adapted from the multi-robot pickup-and-delivery task in [26], describes a delivery task. Each action proposition a_i is associated with an execution requirement $c(a_i) = (\tau_i, n_i, l_i)$, where τ_i is the required robot type, n_i is the required number of robots, and l_i is the target region. Specifically, $c(a_1) = (2, 3, l_1)$, $c(a_2) = (1, 2, l_8)$, $c(a_3) = (1, 1, l_2)$, $c(a_4) = (2, 3, l_4)$, $c(a_5) = (1, 3, l_7)$, $c(a_6) = (1, 2, l_6)$, $c(a_7) = (1, 3, l_7)$, $c(a_8) = (1, 3, l_8)$, $c(a_9) = (1, 3, l_5)$, $c(a_{10}) = (1, 3, l_3)$, $c(a_{11}) = (1, 3, l_1)$.

In Table III, we report the cost and computation time of the first feasible plan, the plan-graph best, and the global optimum. We also report the sizes of the GBA before and after pruning, denoted by $|\mathcal{G}_\varphi|$ and $|\mathcal{G}_\varphi^-|$, respectively, and the size of the resulting plan graph, denoted by $|G_P|$. Each GBA size is given as the number of states and transitions, whereas the plan-graph size is given as the number of plan nodes and edges.

As shown in Table III, GBA pruning substantially reduces the number of transitions for all four specifications, especially for φ_1 , φ_3 , and φ_4 . Consequently, fewer transitions are propagated to plan-graph construction, reducing both computation and memory usage. The first feasible plan has the same cost as the plan-graph best for φ_1 and φ_2 . For φ_3 and φ_4 , its cost is only 1.83% and 4.47% higher, respectively. Moreover, the costs of the first feasible plans remain close to the corresponding global optima for all four specifications. During plan-graph construction, the residual-obligation score orders candidate successors according to the current plan cost and the estimated travel required for the remaining tasks. By considering lower-score successors first, the method tends to reach a low-cost accepting branch early. Consequently, it can return a high-quality feasible plan without waiting for the complete plan graph to be constructed.

D. Scalability

This experiment evaluates how the proposed method scales as the number of robots and task complexity increase. Since poset-based method [3] and MILP-based method [7] are representative state-of-the-art approaches to LTL-MRTA, we compare the proposed method with these approaches to evaluate its scalability as the number of robots and task complexity increase. The specifications describe warehouse cargo-handling

TABLE II
EFFECT OF GBA-LEVEL PRUNING

m	Proposed			Case II		
	$ \mathcal{B}_\varphi $	$ \mathcal{B}_\varphi^- $	T_{first} (s)	$ \mathcal{B}_\varphi $	$ \mathcal{B}_\varphi^- $	T_{first} (s)
12	(4 096, 28 672)	(4 096, 28 672)	10.4065	(4 096, 61 440)	(4 096, 28 672)	13.5997
13	(8 192, 61 440)	(8 192, 61 440)	30.7849	(8 192, 143 360)	(8 192, 61 440)	68.6376
14	(16 384, 131 072)	(16 384, 131 072)	156.0395	(16 384, 327 680)	(16 384, 131 072)	239.8623
15	(32 768, 311 296)	(32 768, 311 296)	723.6905	(32 768, 737 280)	(32 768, 311 296)	1235.5359

Note: $|\mathcal{B}_\varphi|$ denotes the NBA size when NBA generation is completed, whereas $|\mathcal{B}_\varphi^-|$ denotes the retained size after pruning. For the proposed method, pruning is completed at the GBA level before NBA generation; hence, the generated NBA already contains only the retained transitions and the two sizes are identical. Each NBA size is reported as $(|Q_B|, |\rightarrow_B|)$.

TABLE III
QUALITY OF THE FIRST FEASIBLE PLAN

Specification	First Feasible Plan		Plan-Graph Best		Global Optimum		$ \mathcal{G}_\varphi $	$ \mathcal{G}_\varphi^- $	$ G_P $
	Cost	Time (s)	Cost	Time (s)	Cost	Time (s)			
φ_1	125.23	0.1702	125.23	0.1702	102.67	1.3557	(144, 2916)	(144, 640)	(622, 1847)
φ_2	74.06	0.0619	74.06	0.0619	67.73	0.1522	(16, 82)	(16, 49)	(35, 57)
φ_3	173.36	0.0807	170.24	0.1071	170.24	0.1071	(36, 324)	(20, 60)	(41, 67)
φ_4	212.98	0.4864	203.87	0.5127	176.93	1.0008	(97, 1044)	(81, 291)	(171, 359)

Note: Cost and time denote plan completion time and computation time, respectively. $|\mathcal{G}_\varphi|$, $|\mathcal{G}_\varphi^-|$, and $|G_P|$ denote the sizes of the GBA before pruning, the pruned GBA, and the plan graph, respectively. Each size is reported as (number of states or nodes, number of transitions or edges).

and safety-inspection tasks of varying complexity:

$$\begin{aligned}
\varphi_1 &= \Diamond a_1 \wedge \Diamond a_2 \wedge \Diamond(a_3 \wedge \Diamond a_4) \wedge \Diamond(a_5 \wedge \neg p_7) \\
&\quad \wedge \Diamond a_6 \wedge \Diamond a_9 \wedge (\Box \neg a_9 \mathcal{U} a_6), \\
\varphi_2 &= \Diamond a_1 \wedge \Diamond a_2 \wedge \Diamond(a_3 \wedge \Diamond(a_6 \wedge \Diamond(a_4 \vee a_5))) \\
&\quad \wedge \Diamond a_7 \wedge \Diamond(a_9 \wedge \Diamond a_{10}), \\
\varphi_3 &= \Diamond(a_1 \wedge \Diamond a_2) \wedge \Diamond a_3 \wedge \Diamond a_6 \wedge \Diamond(a_4 \vee a_5) \\
&\quad \wedge \Diamond a_8 \wedge \Diamond a_7 \wedge \Diamond a_9 \wedge \Diamond a_{10} \wedge (\neg a_9 \mathcal{U} a_2).
\end{aligned}$$

Specifically, $c(a_1) = (1, 2, l_1)$, $c(a_2) = (2, n, l_2)$, $c(a_3) = (2, 1, l_3)$, $c(a_4) = (1, \lfloor n/2 \rfloor, l_4)$, $c(a_5) = (5, \lceil 2n/3 \rceil, l_5)$, $c(a_6) = (3, \max 2, \lceil 0.75n \rceil, l_6)$, $c(a_7) = (1, n, l_7)$, $c(a_8) = (1, 2, l_8)$, $c(a_9) = (3, 1, l_8)$, $c(a_{10}) = (4, 1, l_8)$. A configuration denoted by $5 \times n$ contains five robot types and n robots of each type. For example, let $n = 10$, 5×10 denotes a team with five types of robots and ten robots of each type, i.e., 50 robots in total.

Observe in Table IV that the proposed method outperforms the poset- and MILP-based methods in obtaining both the first feasible solution and the best solution found. As the robot team grows, T_{first} changes only moderately for both the proposed and poset-based methods. Nevertheless, the proposed method remains substantially faster, and its advantage in both T_{first} and T_{best} becomes more evident for complex specifications. Although the MILP-based method completes the exhaustive search sooner for some larger instances, the proposed method obtains and improves executable plans much earlier.

The reason is that the proposed method performs task allocation during NBA generation and returns a feasible plan once an accepting prefix-suffix structure is found. The score-guided plan-graph construction prioritizes promising branches and provides a high-quality initial solution for subsequent BnB optimization. In contrast, the poset-based method must

first extract the partial-order structures, whose computation depends mainly on the task specification. The search space of MILP-based method grows rapidly as the numbers of tasks and robots increase, resulting in a longer time to obtain a feasible solution.

E. Online Task Reallocation Under Robot Unavailability

This case study evaluates online repair when the available robot set changes during execution. The initial LTL specification and robot configuration are the same as those in Sec. XI-A.

During task execution, robot r_{20} becomes unavailable at $t = 20$ s, which triggers replanning. We run the replanning procedure 25 times. The average times required to obtain the first feasible repaired plan and the best repaired plan are 0.045738s and 0.39032s, respectively. As shown in Fig. 3(a), a_5 was initially assigned to robots r_{20} , r_{21} , r_{22} , and r_{24} . After r_{20} becomes unavailable, r_{23} replaces it and travels to l_5 , as shown in Fig. 3(b). The repaired assignment therefore retains the four robots required to execute a_5 .

XII. PHYSICAL EXPERIMENTS

XIII. CONCLUSION

In this work, a novel on-the-fly planning framework has been proposed for temporal-logic task allocation in heterogeneous multi-robot systems. The framework performs task allocation during score-guided NBA construction and uses GBA-level pruning to obtain an executable solution early. A branch-and-bound procedure further improves the solution. In addition, an online replanning strategy reassigns unfinished tasks when robots become unavailable. Theoretical analysis establishes completeness of our algorithm. Numerical and

TABLE IV
SCALABILITY COMPARISON WITH POSET- AND MILP-BASED METHODS

(a) Computation time (s)										
Specification	Team	Proposed			Poset			MILP		
		T_{first}	T_{best}	T_{search}	T_{first}	T_{best}	T_{search}	T_{first}	T_{best}	T_{search}
φ_1	5×5	0.3890	4.2470	157.28	17.921	24.587	174.114	6.5013	11.8756	336.4283
	5×10	0.3089	8.7278	203.29	18.385	18.985	247.815	6.8304	13.9769	742.93
	5×20	0.4203	11.8629	421.68	17.124	24.8491	463.769	13.9042	18.7462	1859.54
φ_2	5×5	0.6987	3.9554	927.48	89.203	91.287	940.062	51.4076	60.8376	632.27
	5×10	0.7216	4.9678	1378.92	89.43	92.984	1305.835	59.7567	62.2364	1198.765
	5×20	1.1497	3.9071	8590.59	90.137	97.8213	8697.43	67.0037	74.0153	2507.36
φ_3	5×5	1.3311	4.5280	9972.67	267.98	268.25	9903.93	349.1159	356.4569	1093.42
	5×10	1.7716	6.8869	—	268.98	269.54	—	371.2282	425.7460	3467.98
	5×20	1.7783	7.0542	—	270.42	271.85	—	483.9699	508.9515	9075.42

(b) Plan cost								
Specification	Team	Proposed		Poset		MILP		
		J_{first}	J_{best}	J_{first}	J_{best}	J_{first}	J_{best}	
φ_1	5×5	38.247	33.446	53.32	33.446	46.82	33.446	
	5×10	35.987	32.58	33.48	32.58	45.32	32.58	
	5×20	44.026	33.578	45.7	33.578	72.89	33.578	
φ_2	5×5	57.773	50.98	62.45	50.98	66.326	50.98	
	5×10	50.168	47.89	48.78	47.89	56.34	47.89	
	5×20	49.987	48.931	49.076	48.931	65.32	48.931	
φ_3	5×5	69.46	57.42	67.64	57.42	71.326	57.42	
	5×10	47.93	35.71	48.96	35.71	84.56	35.71	
	5×20	42.89	33.89	39.67	33.89	44.54	33.89	

Note: The symbol “—” indicates that the exhaustive search was not completed within the prescribed limit.

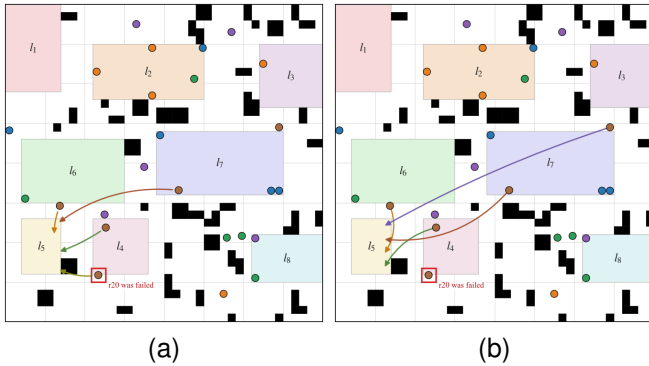


Fig. 3. Online task reallocation after robot r_{20} becomes unavailable: (a) r_{20} fails during task execution; (b) r_{23} replaces r_{20} and travels to l_5 .

physical experiments validate the efficiency, solution quality, scalability, and adaptability of the proposed framework.

ACKNOWLEDGMENTS

This should be a simple paragraph before the References to thank those individuals and institutions who have supported your work on this article.

REFERENCES

- [1] K. Leahy, D. Zhou, C.-I. Vasile, K. Oikonomopoulos, M. Schwager, and C. Belta, “Persistent surveillance for unmanned aerial vehicles subject to charging and temporal logic constraints,” *Autonomous Robots*, vol. 40, no. 8, pp. 1363–1378, 2016.
- [2] B. Li and H. Ma, “Double-deck multi-agent pickup and delivery: Multi-robot rearrangement in large-scale warehouses,” *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3701–3708, 2023.
- [3] Z. Liu, M. Guo, and Z. Li, “Time minimization and online synchronization for multi-agent systems under collaborative temporal logic tasks,” *Automatica*, vol. 159, p. 111377, 2024.
- [4] C. Baier and J.-P. Katoen, *Principles of model checking*. MIT Press, 2008.
- [5] P. Schillinger, M. Bürger, and D. V. Dimarogonas, “Decomposition of finite LTL specifications for efficient multi-agent planning,” in *Distributed Autonomous Robotic Systems: The 13th International Symposium*. Springer, 2018, pp. 253–267.
- [6] L. Li, Z. Zhou, Z. Chen, H. Wang, Z. Kan, and J. Qin, “A formal framework for reactive heterogeneous multirobot task allocation in uncertain semantic environments,” *IEEE Transactions on Cybernetics*, pp. 1–14, 2026.
- [7] X. Luo and M. M. Zavlanos, “Temporal logic task allocation in heterogeneous multirobot systems,” *IEEE Transactions on Robotics*, vol. 38, no. 6, pp. 3602–3621, 2022.
- [8] K. Leahy, Z. Serlin, C.-I. Vasile, A. Schoer, A. M. Jones, R. Tron, and C. Belta, “Scalable and robust algorithms for task-based coordination from high-level specifications (scratches),” *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2516–2535, 2022.
- [9] A. Ulusoy, S. L. Smith, X. C. Ding, C. Belta, and D. Rus, “Optimality and robustness in multi-robot path planning with temporal logic constraints,” *The International Journal of Robotics Research*, vol. 32, no. 8, pp. 889–911, 2013.
- [10] Y. Kantaros and M. M. Zavlanos, “Stylus*: A temporal logic optimal control synthesis algorithm for large-scale multi-robot systems,” *The International Journal of Robotics Research*, vol. 39, no. 7, pp. 812–836, 2020.

- [11] Z. Chen and Z. Kan, “Real-time reactive task allocation and planning of large heterogeneous multi-robot systems with temporal logic specifications,” *The International Journal of Robotics Research*, vol. 44, no. 4, pp. 640–664, 2024.
- [12] P. Gastin and D. Oddoux, “Fast ltl to büchi automata translation,” in *International Conference on Computer Aided Verification*, 2001.
- [13] Z. Chen, Z. Zhou, S. Wang, J. Li, and Z. Kan, “Fast temporal logic mission planning of multiple robots: A planning decision tree approach,” *IEEE Robotics and Automation Letters*, vol. 9, no. 7, pp. 6146–6153, 2024.
- [14] P. Schillinger, M. Bürger, and D. V. Dimarogonas, “Simultaneous task allocation and planning for temporal logic goals in heterogeneous multi-robot systems,” *The International Journal of Robotics Research*, vol. 37, no. 7, pp. 818–838, 2018.
- [15] M. Guo and D. V. Dimarogonas, “Multi-agent plan reconfiguration under local LTL specifications,” *The International Journal of Robotics Research*, vol. 34, no. 2, pp. 218–235, 2015.
- [16] P. Yu and D. V. Dimarogonas, “Distributed motion coordination for multirobot systems under ltl specifications,” *IEEE Transactions on Robotics*, vol. 38, no. 2, pp. 1047–1062, 2022.
- [17] S. L. Smith, J. Tümová, C. Belta, and D. Rus, “Optimal path planning under temporal logic constraints,” in *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2010, pp. 3288–3293.
- [18] Y. Kantaros and M. M. Zavlanos, “Sampling-based optimal control synthesis for multirobot systems under global temporal tasks,” *IEEE Transactions on Automatic Control*, vol. 64, no. 5, pp. 1916–1931, 2018.
- [19] P. Schillinger, M. Bürger, and D. V. Dimarogonas, “Multi-objective search for optimal multi-robot planning with finite ltl specifications and resource constraints,” in *2017 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2017, pp. 768–774.
- [20] S. Karaman and E. Frazzoli, “Vehicle routing with linear temporal logic specifications: Applications to multi-uav mission planning,” in *AIAA guidance, navigation and control conference and exhibit*, 2008, p. 6838.
- [21] L. Li, Z. Chen, H. Wang, and Z. Kan, “Task allocation of heterogeneous robots under temporal logic specifications with inter-task constraints and variable capabilities,” *IEEE Transactions on Automation Science and Engineering*, vol. 22, no. 000, pp. 14 030–14 047, 2025.
- [22] X. Luo and C. Liu, “Simultaneous task allocation and planning for multi-robots under hierarchical temporal logic specifications,” *IEEE Transactions on Robotics*, 2025.
- [23] Z. Liu, M. Guo, W. Bao, and Z. Li, “Fast and adaptive multi-agent planning under collaborative temporal logic tasks via poset products,” *Research*, vol. 7, p. 0337, 2024.
- [24] P. Brucker, S. Knust, A. Schoo, and O. Thiele, “A branch and bound algorithm for the resource-constrained project scheduling problem,” *European Journal of Operational Research*, vol. 107, no. 2, pp. 272–288, 1998.
- [25] Y. Liu, S. Jin, J. Zhou, and Q. Hu, “A branch-and-bound algorithm for the unit-capacity resource constrained project scheduling problem with transfer times,” *Computers & Operations Research*, vol. 151, p. 106097, 2023.
- [26] X. Luo, S. Xu, R. Liu, and C. Liu, “Decomposition-based hierarchical task allocation and planning for multi-robots under hierarchical temporal logic specifications,” *IEEE Robotics and Automation Letters*, vol. 9, no. 8, pp. 7182–7189, 2024.